

How Modern Software Is Built

The architecture of websites, mobile apps, installed programs, and embedded devices, explained for business students who will be building products with Claude Code and learning Python at the same time.

Audience Business students, little technical background

Format Lecture with live demos and review questions **Sessions** 5 × 75 minutes

How to use this document

- **Each session is designed to fit in 75 minutes.** Every section carries a time budget tag such as **10 MIN**. The budgets add up to about 70 minutes, leaving 5 minutes for the review questions and spill-over.
- **Keywords students should be able to define** are highlighted like **architecture**. Every keyword is collected in Appendix A with a short definition and a link back to where it is introduced.
- **Review questions** appear at the end of each session in a yellow box marked with a question mark. The questions and their answers are repeated in Appendix B.
- **Callouts** in grey are instructor notes, analogies you can use in class, and suggested live demos. Demos are optional; each session works as a pure lecture. Callouts with a teal edge labelled "In the age of AI" say how AI-assisted development changes the point just made.
- **Diagrams** are drawn so that you can talk through them left to right or top to bottom. Each caption states the single point the picture is making.

Software architecture sounds like a topic for engineers, but the decisions it describes are business decisions in disguise. Whether a product can handle a spike of customers on launch day, what it costs to run every month, how quickly a new feature can ship, what happens when a server fails at 2 a.m., and how a customer's password is protected are all consequences of architecture. When you use a tool like Claude Code to build a product, you will be asked to make these decisions, or at least to approve them. This module gives you the vocabulary and the mental models to do that well.

The material is deliberately cumulative. Session 1 asks what software is written in and how that choice is made. Session 2 starts with a web page that is nothing more than a text file on your laptop. By the end of Session 5 you will be able to sketch the architecture of a company like Airbnb or Shopify, explain what "Sign in with Google" actually does behind the scenes, and describe how a phone app, a desktop program, and a smart thermostat all talk to the same servers.

Choosing a Programming Language

Before we look at how software is structured, we look at what it is written in. There are hundreds of programming languages, and the choice among them is one of the first decisions any product team makes and one of the hardest to undo. This session explains what the choice depends on: what each language was designed for, how popular and mature it is, what ready-made building blocks exist for it, how fast it runs, and where it can run. For each factor we ask a new question: how much does it still matter now that an AI assistant can write the code?

BY THE END OF THIS SESSION STUDENTS CAN

- Explain what a programming language is and the difference between a language, a library, and a framework.
- Name the languages typically used for websites, back ends, databases, mobile apps, and devices, and say why SQL is not used to build a website.
- Explain why a language's popularity, maturity, and ecosystem matter to a business.
- Say when execution speed and memory use matter and when they do not.
- Describe how software reaches different platforms: recompiling, virtual machines, interpreters, and per-platform mobile languages.
- State how AI-assisted development changes the weight of each factor.

SECTION	MINUTES
1.1 <u>What a programming language is, and why there are so many</u>	8
1.2 <u>Fit for purpose: what each language was designed to do</u>	12
1.3 <u>Popularity, maturity, and community</u>	10
1.4 <u>The ecosystem: packages and libraries</u>	10
Review questions and buffer	5

SECTION	MINUTES
<u>1.5 Speed and memory: when performance matters</u>	10
<u>1.6 Platforms and portability</u>	12
<u>1.7 Making the decision in the age of AI</u>	8
Review questions and buffer	5

1.1 What a programming language is, and why there are so many

8 MIN

A computer's processor understands only **machine code**: numbers that mean "add these two values," "copy this to memory," "jump to that instruction."

Nobody writes that by hand. A **programming language** is a human-readable notation for those instructions, with a precise grammar (its **syntax**) so that a program can translate it into machine code without ambiguity. What a programmer writes is called **source code**.

Two translation strategies exist, and the difference will matter later in this session:

- A **compiler** translates the whole program ahead of time into an executable file. The result runs very fast, but the file is tied to one kind of processor and operating system. C, C++, Rust, Go, and Swift work this way.
- An **interpreter** reads the source code and carries it out on the spot, line by line. Nothing is built in advance; the same file runs wherever the interpreter is installed. Python, JavaScript, and Ruby work this way. (Java and C# sit in between and get their own explanation in Section 1.6.)

ANALOGY

A compiler is a translator who takes your whole book and produces a finished French edition; readers get it fast, but you need a separate edition for every language. An interpreter is a live interpreter at a meeting who translates each sentence as you say it; slower, but the same speech works in any room where an interpreter is present.

Why are there hundreds of languages instead of one? Because each was created to make a particular kind of work easier, at the cost of making other kinds harder. Some trade speed for ease of writing (Python). Some trade ease for total control of the hardware (C). Some exist because a company controls a

platform and wants a language optimised for it (Swift for Apple, Kotlin for Android). Some are not meant for writing whole programs at all but for one specialised job (SQL for asking questions of a database). Choosing a language is choosing which trade-offs suit your product.

Three words that get mixed up, and a hierarchy to keep them straight:

- A **language** is the notation itself: Python.
- A **library** (also called a package or module) is a bundle of ready-made code in that language that does one job, which you call instead of writing yourself: a library for sending email, one for drawing charts.
- A **framework** is a large, opinionated set of libraries that provides the skeleton of a whole kind of application; you fill in the parts unique to your product. Flask and Django are web frameworks for Python; React is a front-end framework for JavaScript. Session 3 uses one.

IN THE AGE OF AI

The translation layer, the syntax, is the part of programming that AI assistants have most completely absorbed. An assistant such as Claude Code will produce correct syntax in any mainstream language on request, which means *knowing the syntax* of a language is no longer the barrier it was. The trade-offs described in this session, however, are properties of the languages themselves, not of the person typing, and they do not disappear when an AI does the typing. The rest of this session is about those.

1.2 Fit for purpose: what each language was designed to do

12 MIN

The first question is the simplest: was this language built for the job?

Languages divide into **general-purpose languages**, which can write almost any kind of program, and **domain-specific languages**, which are excellent at one narrow thing and useless outside it.

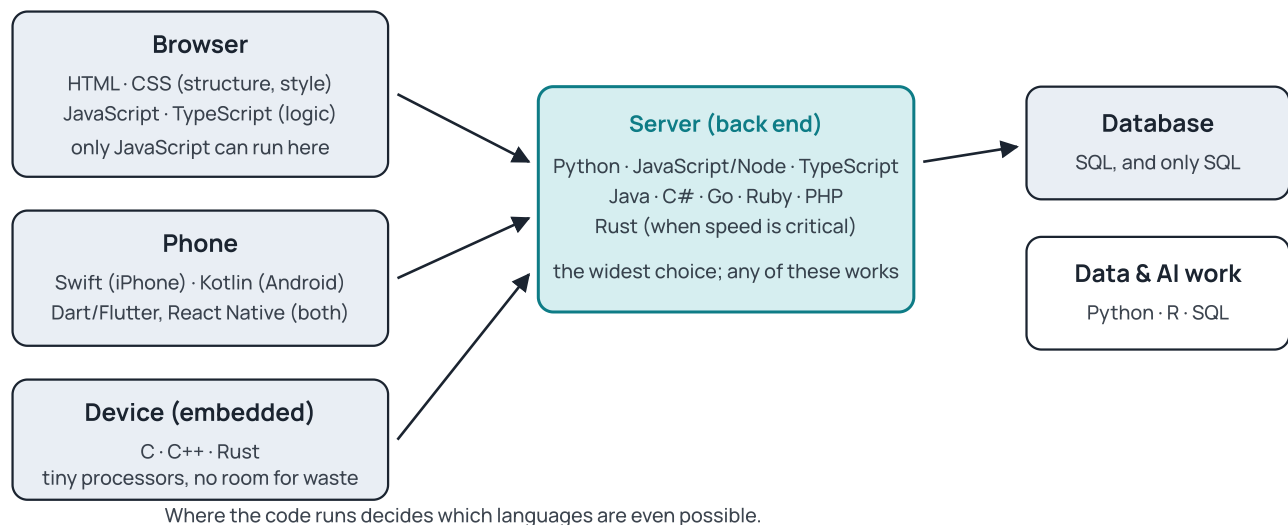
SQL is the clearest example of the second kind. It is superb at describing questions about tables ("all Boston customers who ordered rye twice last quarter") and it is the right tool for that everywhere it comes up. But it has no way to draw a button, listen for a click, send an email, or lay out a page.

Nobody writes a website in SQL for the same reason nobody writes a novel in the notation of chess: the language simply cannot express the ideas. HTML and CSS are also domain-specific: they describe structure and appearance and are not, strictly, programming languages at all, since they cannot express logic such as "if the order is over 50 dollars, show the discount."

Among general-purpose languages the fit is softer but real. Each has a home turf where its design, its tooling, and its community are strongest, and using it elsewhere is possible but uphill. The table maps the languages you will hear about to what they are typically used for.

LANGUAGE	DESIGNED FOR / HOME TURF	TYPICAL USES TODAY	NOTES
Python	Readability and quick development	Back ends, data analysis, AI and machine learning, automation scripts, scientific computing	The language of the AI industry. What you are learning in your other course.
JavaScript	Interactivity inside web pages	All front-end code in browsers; back ends via Node.js; cross-platform mobile and desktop apps	The only language browsers run, which makes it unavoidable for web products.
<u>TypeScript</u>	JavaScript for large projects	Same as JavaScript, with extra checking that catches mistakes before the code runs	Translates into JavaScript; increasingly the default for serious web work.
SQL	Querying relational databases	Every relational database, every report, every dashboard	Domain-specific. Used alongside a general-purpose language, never instead of one.
Java	Large, long-lived business systems that run anywhere	Banking and enterprise back ends, Android apps (older ones), big-data tools	Verbose but extremely mature; huge corporate installed base.
C#	Microsoft's answer to Java	Windows software, enterprise back ends (.NET), games (Unity)	Strong in companies already invested in Microsoft tools.
C and C++	Total control of hardware and speed	Operating systems, game engines, browsers, embedded devices, high-frequency trading	Fastest, hardest to write safely. Most of the software under everything else.
Rust	C++ speed without C++'s safety	Systems software, performance-critical	Newer (2015); growing fast; steeper learning

LANGUAGE	DESIGNED FOR / HOME TURF	TYPICAL USES TODAY	NOTES
	problems	services, increasingly embedded	curve.
Go	Simple, fast network services at Google scale	Cloud infrastructure, back- end services, command-line tools	Docker and Kubernetes (Session 4) are written in it.
Swift	Apple platforms	iPhone, iPad, Mac apps	Apple's preferred language; the best experience on its devices.
Kotlin	Android	Android apps; some back ends	Google's preferred Android language since 2019.
R	Statistics	Statistical analysis, academic research, some finance	Beloved by statisticians; rarely used to build products.
PHP, Ruby	Building websites quickly (1990s– 2000s)	WordPress and much of the older web (PHP); Shopify, GitHub, Airbnb's early code (Ruby on Rails)	Mature, still widely deployed, less often chosen for new products.



Where each language lives. Some places admit only one language (the browser, the database). The server admits almost any, which is why the back-end choice is the one most worth thinking about.

Notice a pattern in the diagram: a modern product is never written in one language. A typical web product uses HTML, CSS, and JavaScript in the browser, Python or JavaScript on the server, and SQL in the database, with Swift and Kotlin if it has native apps. "Choosing a language" really means choosing the back-end language and, if relevant, the mobile strategy; the rest is dictated by where the code has to run.

IN THE AGE OF AI

Fit for purpose matters exactly as much as before. An AI assistant cannot make SQL draw a button or make a browser run Python; those are facts about the platforms. What AI changes is the cost of using several languages in one product. A founder used to avoid a language they did not know; now the assistant handles the Swift for the iPhone app and the SQL for the reports while the founder reviews the results. The number of languages in a product is less of a staffing constraint than it was, but each one still has to be the right one for its layer.

1.3 Popularity, maturity, and community 10 MIN

Two languages can be equally capable on paper and yet one is a far better business choice, because a language is not just a notation. It is a community of people using it, a body of documentation and answered questions, a supply of developers who can be hired, and a track record of being maintained. Together these are the language's **maturity** and popularity, and they matter for concrete reasons:

- **Hiring.** A product written in a popular language can be worked on by millions of developers; one written in an obscure language depends on a few thousand, who cost more and take longer to find. When the original developer leaves, this becomes urgent.
- **Answers.** Every problem a developer hits in a popular language has been hit before, written up, and solved somewhere public. In a niche language, they are on their own.
- **Longevity.** Software lasts longer than anyone plans. Systems written in COBOL in the 1970s still run banks and governments, and the language's small pool of programmers is now a well-known crisis. A language with a large base is far more likely to still be supported, patched, and running on new hardware in twenty years.
- **Security.** Popular languages and their libraries are inspected by many eyes, and flaws are found and fixed quickly. Obscure ones can carry unfixed problems for years.
- **Tooling.** Editors, debuggers, testing tools, hosting platforms, and cloud services all support the popular languages first and best.

Popularity is measured in several imperfect ways: surveys of developers, counts of questions asked online, job postings, and activity on GitHub. They disagree on details but agree on the picture: Python, JavaScript/TypeScript, Java, C#, C/C++, Go, and SQL are the mainstream, with Rust, Swift, and Kotlin strong in their niches. A business choosing from that set is choosing safely.

The other side of maturity is **age**. A very new language may be genuinely better designed than its predecessors and still be the wrong choice, because it has few libraries, changes in ways that break existing code, and has not yet shown it will survive. Most successful languages needed ten years to become safe bets. The cautious rule: let others be early adopters unless the new language solves a problem you actually have.

IN THE AGE OF AI

Popularity matters more than before, for a new reason. AI coding assistants learn from the enormous body of public code and documentation on the internet. The more a language has been written and discussed, the better the assistant is at writing it: Python, JavaScript, TypeScript, Java, and SQL get outstanding support; a niche or very new language gets noticeably weaker, more error-prone help. The same applies to versions: an assistant trained mostly on last year's version of a fast-changing framework will confidently produce last year's code. So the hiring argument for mainstream languages has weakened a little (AI can cover a skills gap), but

the "quality of AI help" argument has more than replaced it. If you are going to build with AI, build in the languages the AI knows best, on stable, well-documented versions.

1.4 The ecosystem: packages and libraries 10 MIN

No serious program is written from scratch. Sending an email, reading a spreadsheet, charging a credit card, resizing an image, talking to a database, drawing a chart: for any language worth using, someone has already written each of these and published it for free. A language's **ecosystem** is that collection of ready-made parts plus the tools for finding and installing them, and it is frequently the deciding factor between two otherwise similar languages.

The mechanics are worth knowing because they come up daily. Libraries are distributed through a **package manager**, which fetches them from a central registry and keeps track of which versions your project uses. Python has **pip** (and a newer, faster tool called **uv**) drawing from a registry called PyPI with over half a million packages. JavaScript has **npm**, with more than two million. When a developer says "we added a dependency," they mean the project now relies on someone else's package.

Almost all of these packages are **open source**: published freely for anyone to use, inspect, and improve. Open source is why a two-person startup can build on the same foundations as a bank. It is also a supply chain. A package you depend on may itself depend on fifty others, each maintained by volunteers who may stop, and occasionally one is abandoned or, rarely, deliberately sabotaged. Mature ecosystems have tooling that flags known-vulnerable packages, which is one more reason maturity matters.

Ecosystem strength is uneven across domains, and this is where it steers language choice:

IF YOUR PRODUCT IS MOSTLY ABOUT...	THE ECOSYSTEM PULLS YOU TOWARD	BECAUSE
AI, machine learning, data analysis	Python	Every major AI library (PyTorch, scikit-learn, pandas, the Claude and OpenAI SDKs) is Python-first. Other languages get ports later, if at all.
A rich, interactive web front end	JavaScript / TypeScript	React, Vue, and the entire browser tooling world live here, and nothing else runs in the

IF YOUR PRODUCT IS MOSTLY ABOUT...	THE ECOSYSTEM PULLS YOU TOWARD	BECAUSE
		browser.
A conventional web back end with a database	Python, JavaScript, Ruby, Java, C#, Go, PHP	All have mature web frameworks; the choice is made by the other factors and by team familiarity.
Large enterprise integration (SAP, Oracle, mainframes)	Java, C#	Decades of connectors and vendor support.
Games	C# (Unity), C++ (Unreal)	The engines dictate the language.
A native iPhone or Android app	Swift / Kotlin	Apple and Google build their libraries and tools for these first.

IN THE AGE OF AI

The ecosystem matters as much as ever, and there is a new failure mode to watch. An assistant is only as capable as the libraries it can call; it will not write a payment processor from scratch any more than a human would, so a language with the right packages is still the right language. Two things change. First, the assistant knows the ecosystem better than any individual, and will reach for the standard library for a job you did not know had one, which makes rich ecosystems even more valuable. Second, assistants sometimes **hallucinate**: they confidently name a package or function that does not exist, or that existed in an older version. The install fails or the code will not run, which makes it easy to catch, but it is why the person directing the assistant still needs to check that a suggested dependency is real, maintained, and reputable before it goes into a product.

1.5 Speed and memory: when performance matters

10 MIN

Languages differ enormously in how fast the programs written in them run and how much memory they use. Compiled languages such as C, C++, and Rust are typically ten to a hundred times faster than interpreted ones such as Python at raw computation, and use a fraction of the memory, because there is no interpreter in the way and the programmer controls exactly how data is laid out. Java, C#, and Go sit between the two.

The instinct is to want the fastest, and it is usually wrong. What matters is *where the time actually goes*:

- **Most business applications wait, they do not compute.** A web request spends its time waiting for the database, the network, and the disk. Whether the surrounding logic runs in one millisecond or ten is invisible next to a fifty-millisecond database query. Instagram, Dropbox, Spotify, and Reddit serve enormous audiences from Python and the like, because the architecture in Sessions 4 and 5 (caching, more servers) buys far more speed than a faster language would.
- **Developer time costs more than machine time.** A language that lets a team ship in a week instead of a month is worth more than one that saves a few servers, at almost any scale short of the very largest. Servers cost dollars; developers cost thousands.
- **The slow language often calls the fast one.** Python's data and AI libraries are Python on the outside and C or C++ on the inside. The heavy arithmetic runs at compiled speed; the developer writes the convenient language. This is why "Python is slow" and "Python runs the world's AI" are both true.

Performance genuinely decides the language in a minority of cases, and they are recognisable:

SITUATION	WHY PERFORMANCE WINS	TYPICAL CHOICE
Embedded devices, wearables, sensors	Kilobytes of memory, a slow processor, a battery to preserve. Every byte and cycle counts.	C, C++, Rust
Games and graphics, video processing	Sixty frames a second leaves 16 milliseconds per frame for everything.	C++, C# (Unity), Rust
High-frequency trading, real-time control	Microseconds are money or safety.	C++, Rust
Very large scale services	At millions of requests per second, a 10× efficiency difference is thousands of servers.	Go, Rust, Java, C++ for the hot parts
Mobile apps	Battery life and a phone's memory limits; users notice a sluggish app instantly.	Swift, Kotlin (compiled for the device)

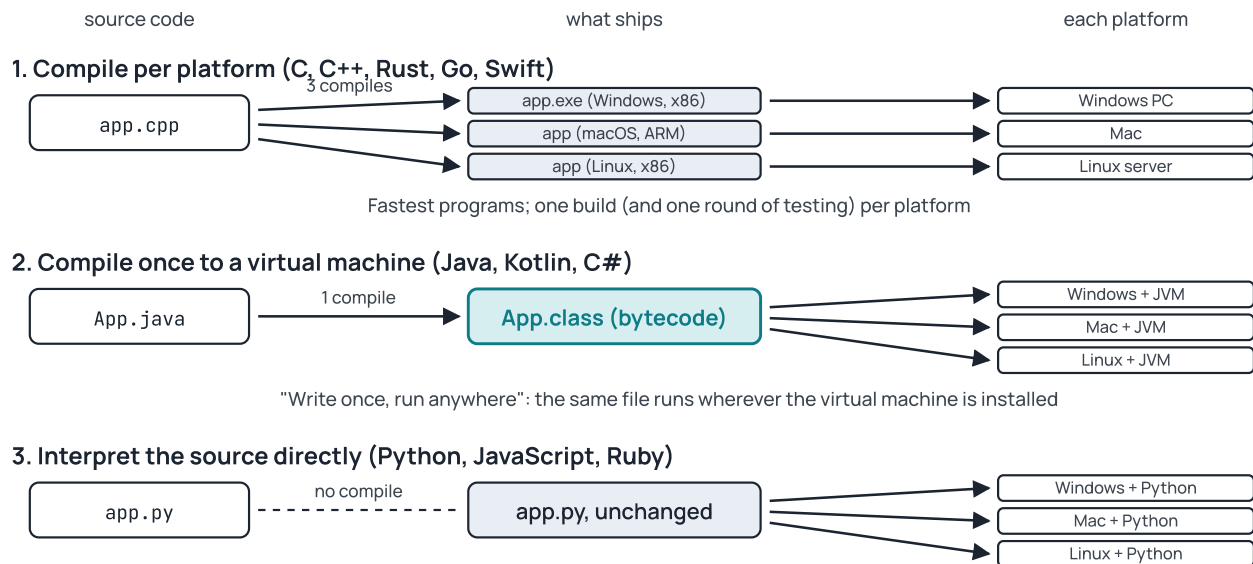
Memory footprint follows the same logic: it decides everything on a device with 256 kilobytes and almost nothing on a server with 64 gigabytes. The sensible sequence for most products is to build in the productive language, measure, and rewrite only the specific slow part in a faster one if measurement proves it necessary. Teams that guess in advance which part will be slow are usually wrong.

IN THE AGE OF AI

Performance constraints are physics and do not move; the cost of responding to them drops sharply. A thermostat's processor is no faster because an AI wrote the code, and a Python loop is no faster either. But the traditional reason to *start* in a fast, hard language, that rewriting later would be too expensive, has weakened. An assistant can take a slow Python function that measurement has identified and produce a Rust or C++ equivalent in minutes, a job that once justified hiring a specialist. The practical rule becomes even more strongly "build in the productive language, measure, then optimise the hot spots," because the last step is now cheap. Where performance rules from the start (embedded, games, trading), AI assistance makes the demanding languages more accessible to a small team but does not change which languages those are.

1.6 Platforms and portability 12 MIN

A **platform** is the combination of processor and operating system a program runs on: Windows on an Intel laptop, macOS on an Apple chip, Linux on a cloud server, iOS on an iPhone, Android on a Samsung phone. Machine code for one is gibberish to another, for two independent reasons. The **processor architecture** differs (Intel and AMD chips speak x86; phones, Apple silicon Macs, and a growing share of servers speak ARM), and the operating system differs in how programs ask for files, windows, and the network. Getting the same software onto several platforms is a real cost, and languages handle it in three different ways.



Three ways to reach three platforms. Compiled languages produce a separate program per platform. Java-style languages compile once to an intermediate form that a virtual machine runs anywhere. Interpreted languages ship the source itself. Each step down trades speed for portability.

- 1. Recompile for each platform.** C and C++ programs are compiled straight to machine code, so a Windows build, a Mac build, and a Linux build are three separate products, each needing its own compilation and its own testing. When Apple moved Macs from Intel to its own ARM chips in 2020, every compiled Mac program had to be rebuilt, which is why some software ran slowly in an emulator for a year or two. Rust, Go, and Swift work the same way, though their tooling makes the rebuilds easier.
- 2. Compile once, run on a virtual machine.** Java was designed in the 1990s around the slogan "write once, run anywhere." Java source is compiled to bytecode, a machine code for an imaginary computer, and a virtual machine (the JVM) installed on each real platform runs that imaginary computer. The same `.class` file runs unchanged on Windows, Mac, Linux, and, historically, on phones and smart cards. Kotlin, Scala, and Clojure share the JVM; C# and the other .NET languages use the equivalent .NET runtime. The price is the virtual machine itself: it must be installed, it uses memory, and it takes a moment to start.
- 3. Ship the source and interpret it.** Python, JavaScript, and Ruby are never compiled by the developer at all. The same source file runs anywhere the interpreter is installed. JavaScript is the extreme case: the interpreter is built into every browser on earth, which makes the web the most portable platform that has ever existed. A web app needs no per-platform version, no app store, and no installer.

Mobile phones are the important special case. Apple and Google each built their platform around a preferred language and toolset: Swift (and its predecessor Objective-C) for iOS, Kotlin (and older Java) for Android. Both are compiled for the phone's processor, and the platform's own libraries, design conventions, and store tooling are built for them first. A company that wants the best possible app on both platforms therefore writes it twice. [Cross-platform frameworks](#) such as React Native (JavaScript) and Flutter (Dart) exist to avoid that: one codebase produces both apps, at some cost in performance and in fidelity to each platform's conventions. Session 5 returns to this trade-off in detail. For desktop software, Electron plays the same role, wrapping a web app so that Slack, VS Code, and Discord ship one codebase to Windows, Mac, and Linux.

The business reading of all this: every additional platform is an additional product to build, test, and support, and the language strategy determines how much of that work is duplicated. The web minimises it, virtual-machine languages nearly eliminate it for desktop and server software, compiled languages maximise it but run fastest, and native mobile doubles it on purpose in exchange for the best user experience.

IN THE AGE OF AI

Portability matters somewhat less as a language-selection factor, and just as much as a testing burden. The reason to fear multiple platforms was the labour of writing and maintaining several versions. An assistant can port a function from Swift to Kotlin, adjust a build for ARM, or generate the platform-specific glue in a fraction of the time, so "we would have to write it twice" is a weaker objection than it was. What AI does not do is run the app on a real iPhone, a real Samsung, and a real Windows laptop and notice that the layout breaks or the battery drains. Each platform still has to be tested and supported by people. So choose the platform strategy on the basis of what your users need and what you can test, and let AI absorb more of the duplicated writing.

1.7 Making the decision in the age of AI 8 MIN

[AI-assisted development](#) changes who types the code but not which language the code should be in. Pulling the session together, here is how each factor shifts:

FACTOR	THE QUESTION TO ASK	WEIGHT BEFORE AI	WEIGHT WITH AI ASSISTANCE
Fit for purpose	Was this language built for this layer and this job?	Decisive	Unchanged. Platforms dictate the browser, database, and mobile languages; AI cannot override that.
Popularity and maturity	How many people use it, how long has it been stable, will it exist in ten years?	High (hiring, answers, longevity)	Higher. AI quality tracks how much public code exists. Mainstream, stable languages get the best help.
Ecosystem	Do the libraries I need already exist and are they maintained?	High	Unchanged to higher. AI uses libraries rather than replacing them; verify that suggested packages are real.
Speed and memory	Does measurement say the language is the bottleneck, or is it the database and network?	Decisive in a minority of domains; overrated elsewhere	Same domains decisive; elsewhere lower. Rewriting a hot spot in a fast language is now cheap, so start productive.
Platforms	Where must this run, and how many separate versions does that imply?	High (duplicated work)	Lower for writing, unchanged for testing. AI ports code; people still verify every platform.
Your own familiarity	Can I (or my team) read and maintain it?	Often the deciding factor in practice	Lower but not zero. AI lets you build in a language you do not know; you still have to review what it wrote and understand it when it breaks.

Three conclusions follow, and they are the ones to carry into the rest of the module:

1. **Choose mainstream.** For anything built with an AI assistant, prefer the languages with the largest bodies of public code: Python, JavaScript/TypeScript, SQL, and Java or C# where the enterprise requires them, with Swift and Kotlin for native mobile. This is where the assistant is most reliable and where a human reviewer can most easily find help.

2. **Optimise for readability, not typing speed.** When an AI writes most of the code, the human's job becomes reading it: reviewing, testing, and deciding. A language that is easy to read (Python was designed for exactly this) makes that job possible for a non-specialist. This is a real argument for Python as a first back-end language for people in this course.
3. **The language is rarely the risk.** Products fail far more often from unclear requirements, wrong data models, and missing security than from choosing Python over Go. Spend the decision-making effort on the architecture in Sessions 2 to 5, pick a sensible mainstream language, and move on.

THE STACK USED IN THIS MODULE

The rest of these sessions use HTML, CSS, and JavaScript in the browser, Python on the server, and SQL in the database. That is the combination most compatible with your Python course, the one with the deepest AI support, and one that real companies run at scale. Where a phone app appears, Swift and Kotlin are named as the native choices.

?

REVIEW QUESTIONS

Session 1: Choosing a programming language

1. Explain the difference between a compiler and an interpreter, and name one language of each kind.
2. Why would nobody build a website in SQL, even though SQL is an excellent language?
3. Give three reasons a language's popularity matters to a business, then explain the new reason it matters in the age of AI.
4. What is a package manager, and what is a hallucinated package?
5. A colleague insists your new online store must be written in C++ "because it is the fastest language." How do you respond?
6. Describe the three ways software reaches Windows, Mac, and Linux, and name a language for each.
7. Why do companies often write their mobile app twice, and what alternative exists?
8. Which factor in language choice becomes *less* important with AI-assisted development, which becomes *more* important, and why?

Answers are in [Appendix B, Session 1](#).

From a File on Your Laptop to a Page on the Internet

We start with the simplest possible website, a single text file opened in a browser, and follow it all the way to being served to strangers over the internet. Along the way we meet the browser, the three languages of a web page, addresses, servers, and the request-and-response conversation that every website depends on.

BY THE END OF THIS SESSION STUDENTS CAN

- Explain the client-server model and identify the client and server in everyday examples.
- Describe what HTML, CSS, and JavaScript each contribute to a web page.
- Trace what happens between typing a web address and seeing a page, including DNS and HTTP.
- Distinguish a static website from a dynamic one and explain the limits of static sites.
- Say where Python runs in a web application and why it does not run in the browser.

SECTION	MINUTES
<u>2.1 Why architecture is a business topic</u>	5
<u>2.2 Clients, servers, and programs</u>	10
<u>2.3 A web page is three languages</u>	15
<u>2.4 The static website on your own computer</u>	5
<u>2.5 How the internet delivers a page</u>	20
<u>2.6 Front end, back end, and the limits of static sites</u>	10
<u>2.7 Where Python fits</u>	5
Review questions and buffer	5

2.1 Why architecture is a business topic 5 MIN

When an architect designs a building, they decide where the load-bearing walls go, how people move between floors, and where the plumbing runs. Change those decisions after the concrete is poured and the cost is enormous. Software has the same property. **Architecture** is the set of big structural decisions about a piece of software: what its major parts are, what each part is responsible for, and how the parts talk to each other.

Three reasons a business student should care:

- **Cost.** Architecture decides whether you pay a few dollars a month or tens of thousands. It decides whether adding customers adds cost linearly or barely at all.
- **Risk.** Architecture decides what happens when a component fails, how a data breach could occur, and whether the product survives a viral moment or collapses under it.
- **Speed.** Architecture decides how fast a team (or you with an AI coding assistant) can add features without breaking what already works.

You will not be writing the load-bearing walls yourself in this course. But you will be talking to people and tools that do, and the person who cannot follow the conversation does not get a vote.

INSTRUCTOR NOTE

A good opener is to ask the class to name a website or app that "went down" recently and speculate why. Most outages trace back to a component in this module: a database, a server that ran out of capacity, a login system, a third-party service. Return to their examples as the relevant pieces appear.

2.2 Clients, servers, and programs 10 MIN

Almost every piece of modern software follows one pattern: one computer asks, another computer answers. The computer that asks is the client. The computer that answers is the server. Together this is the client-server model.

Two things students often get wrong about servers:

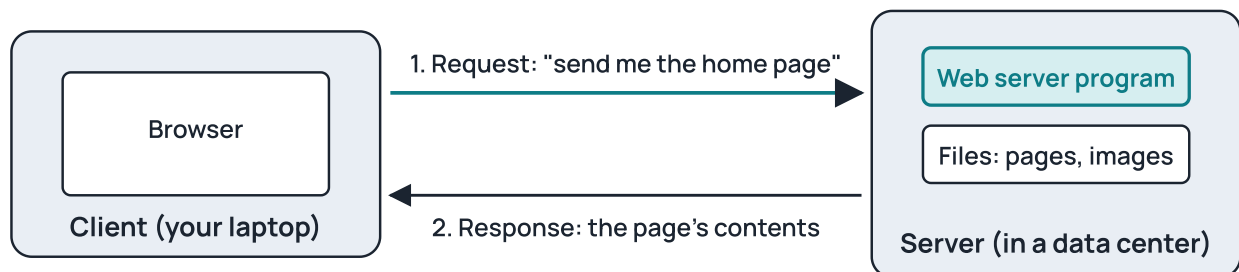
- **A server is an ordinary computer.** It has a processor, memory, and a disk exactly like a laptop. It usually has no screen or keyboard because nobody sits at it. It lives in a building full of other servers (a data center) with

reliable power, cooling, and a fast internet connection. It runs 24 hours a day, waiting.

- "Server" also names a program. A **web server** is a program whose only job is to listen for requests and hand back web pages. The same physical computer can run several server programs at once. In conversation, "the server" means whichever of the two is obvious from context.

ANALOGY

A restaurant. You (the client) sit at a table and ask the waiter for a menu. The kitchen (the server) prepares what you ask for and sends it out. You never enter the kitchen and you don't need to know how it is organized to place an order. The menu is the agreed-upon list of what you may ask for. We will reuse this restaurant repeatedly; the menu becomes the API in Session 3.



Millions of clients can talk to the same server. The server never starts the conversation.

The client-server model. The client always speaks first. The server waits, answers, and forgets about the client until the next request arrives.

Everyday examples students already use, with the client and server identified:

ACTIVITY	CLIENT	SERVER
Reading news on your phone	The browser or news app on the phone	The newspaper's computers that hold the articles
Sending a WhatsApp message	The WhatsApp app	Meta's messaging servers, which pass the message to your friend's phone (another client)
Checking your bank balance	The banking app	The bank's servers holding account records

ACTIVITY	CLIENT	SERVER
Asking Claude a question	The Claude app or Claude Code on your computer	Anthropic's servers running the AI model

The client for a website is nearly always a [web browser](#): Chrome, Safari, Firefox, Edge. The browser is itself just a program installed on your computer. Its job is to request pages and turn what comes back into pictures and text on the screen.

2.3 A web page is three languages 15 MIN

Here is a surprising fact: a web page is a plain text file. You can open it in Notepad. If you save a text file with the name `index.html`, type a few lines into it, and double-click it, your browser will display it as a web page. No internet connection, no server, no company involved.

The text inside that file is written in up to three languages, each with a separate job.

HTML: the content and its structure

[HTML](#) (HyperText Markup Language) says *what is on the page*: this is a heading, this is a paragraph, this is an image, this is a link to another page. It does this with [tags](#), labels in angle brackets that wrap the content.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Rosenthal's Bakery</title>
  </head>
  <body>
    <h1>Rosenthal's Bakery</h1>
    <p>Fresh bread every morning since 1998.</p>
    
    <a href="menu.html">See the menu</a>
  </body>
</html>
```

Read it aloud and it almost explains itself: `<h1>` is a top-level heading, `<p>` is a paragraph, `<img>` places an image stored in a file called `storefront.jpg`, and `<a href=...>` is a link (an "anchor") to another page called `menu.html`.

HTML alone gives you a working but ugly page: black text on white, default fonts, no layout.

CSS: how it looks

CSS (Cascading Style Sheets) says *what the page looks like*: colors, fonts, spacing, how things are arranged, what happens on a phone screen versus a desktop monitor. A CSS rule names the thing it applies to and lists the properties to change.

```
h1 {  
  color: #7a3e12;  
  font-family: Georgia, serif;  
}  
p {  
  font-size: 18px;  
  line-height: 1.6;  
}
```

This says: every top-level heading is a warm brown in the Georgia font; every paragraph uses 18-pixel text with comfortable line spacing. The same HTML file with a different CSS file can look like a completely different website. That separation is deliberate: content people write the HTML, design people write the CSS, and neither steps on the other.

JavaScript: what it does

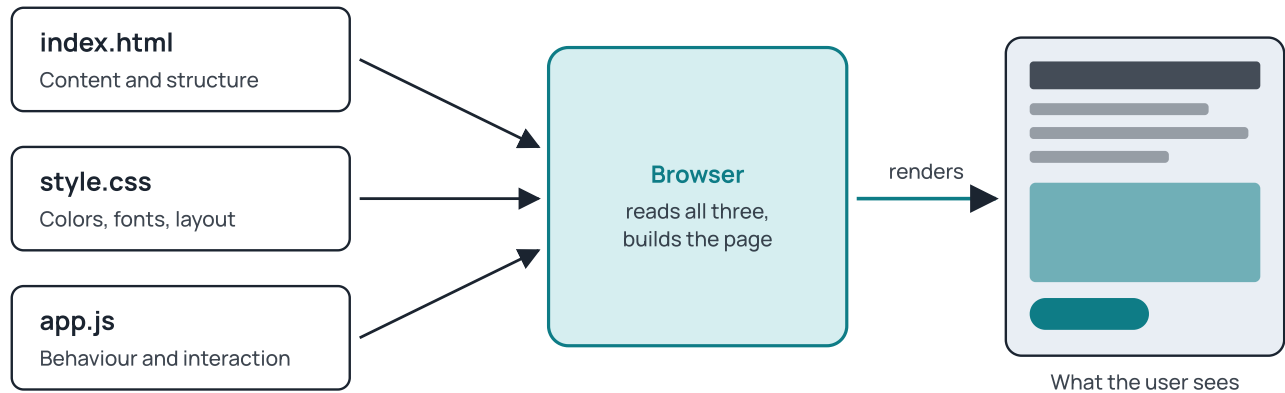
JavaScript is a real programming language, the only one that browsers can run, and it makes the page *do things*: react when a button is clicked, check that an email address looks valid before a form is sent, show a dropdown menu, load more posts as you scroll. Without JavaScript a page is a document. With it, a page becomes an application.

```
const button = document.querySelector("#order-button");  
button.addEventListener("click", function () {  
  alert("Thanks! Your order is on its way.");  
});
```

In words: find the element on the page whose id is `order-button` ; when it is clicked, show a message. Students in a Python course will notice that this looks like a programming language because it is one. The syntax differs from Python (curly braces, semicolons) but the ideas are the same: variables, functions, conditions, loops.

COMMON CONFUSION

JavaScript and Java are unrelated languages that share half a name for 1990s marketing reasons. Java runs on servers and in Android apps. JavaScript runs in browsers (and, since 2009, on servers too, through a program called Node.js). When people say "JS" they mean JavaScript.



Three files, one page. The browser combines structure (HTML), appearance (CSS), and behaviour (JavaScript) into the page a person sees. All three are plain text files.

The browser's act of turning those text files into pixels is called **rendering**. Every browser does it slightly differently, which is why the same site can look marginally different in Safari and Chrome.

2.4 The static website on your own computer 5 MIN

A website made only of files like the ones above, files that are written once and sent to every visitor exactly as they are, is a **static website**. The word "static" means the content does not change depending on who is looking or when. A brochure site for a bakery, a personal portfolio, a documentation site, and a landing page are all naturally static.

The essential point for this session: a static website needs no server at all to work on your own computer. The browser reads the files straight off your disk. This is the zero-cost, zero-infrastructure starting point from which everything else in this module is built up.

LIVE DEMO (3 MINUTES)

1. Create a folder on the desktop called `bakery` .
2. Ask Claude Code: "Create a one-page static website for a bakery in this folder with an `index.html`, a `style.css`, and a small script that shows a greeting when a button is clicked."
3. Double-click `index.html` . Point out the address bar: it starts with `file:///` , meaning "a file on this machine," not `https://` .
4. Change one word in the HTML in a text editor, save, and refresh the browser to show the connection between the file and the page.

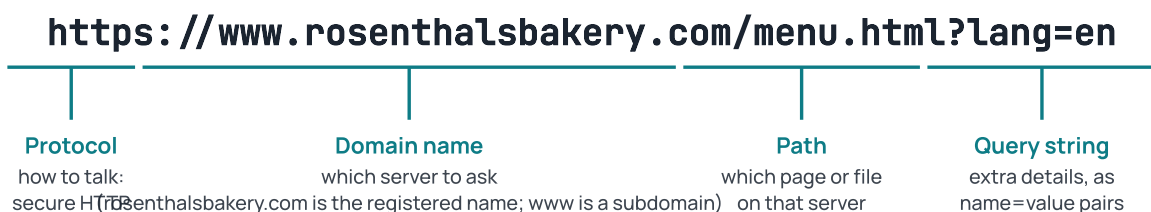
Notice the limitation, though. Only you can see this page. Your friend across town cannot type anything into their browser that reaches a file on your desktop. To share it with the world, the files have to be placed on a server that is always on and reachable by address. That is the subject of the next section.

2.5 How the internet delivers a page 20 MIN

Suppose the bakery's files are now sitting on a server somewhere. A customer types `https://www.rosenthalsbakery.com/menu.html` into their browser. In the roughly half a second before the page appears, a lot happens. We take it piece by piece.

The address: anatomy of a URL

The thing you type is a **URL** (Uniform Resource Locator). It has parts, and each part answers a different question.



Anatomy of a URL. Protocol, domain, path, and optional query string. The domain picks the server; the path picks the file or page on it.

- The **domain name** is the human-friendly name a business rents (typically 10 to 20 dollars a year from a *registrar* such as GoDaddy, Namecheap, or

Cloudflare). Owning the domain is owning the brand's address on the internet.

- The **path** identifies a specific page or file on that server. For a static site the path is literally a file name. For dynamic sites (Session 3) the path is more like an instruction: `/orders/1234` means "show me order number 1234".
- The **query string** passes extra details, like a language or a search term. When you search Google, the words you typed appear in the query string of the results URL.

Finding the server: IP addresses and DNS

Computers on the internet do not find each other by name. Every connected machine has an **IP address**, a number such as `142.250.72.14`, and traffic is routed by number. Nobody wants to memorize numbers, so the internet has a distributed phone book called **DNS** (Domain Name System). When you type a domain, your browser first asks DNS "what is the IP address of `www.rosenthalsbakery.com`?" and gets back a number. Only then does it know where to send the request.

When a business buys a domain and then signs up with a hosting company, "pointing the domain at the host" means editing a DNS record so that the name resolves to the host's IP address. It is one of the few technical tasks a founder often does personally, and it is where the phrase "DNS changes can take up to 48 hours to propagate" comes from: the phone book is copied to thousands of servers worldwide and they refresh on a schedule.

The conversation: HTTP requests and responses

Now the browser knows the number. It opens a connection to that server and speaks a language called **HTTP** (HyperText Transfer Protocol). A **protocol** is simply an agreed set of rules for how two computers exchange messages, the way a business letter has a conventional layout. HTTP is refreshingly readable. The browser's **request** looks like this:

```
GET /menu.html HTTP/1.1
Host: www.rosenthalsbakery.com
User-Agent: Mozilla/5.0 (Macintosh) Chrome/128.0
Accept-Language: en-US
```

Read it as: "GET me the thing at `/menu.html`. I'm looking for the site called `www.rosenthalsbakery.com`. I'm Chrome on a Mac, and I prefer English." The server's **response** is equally readable:

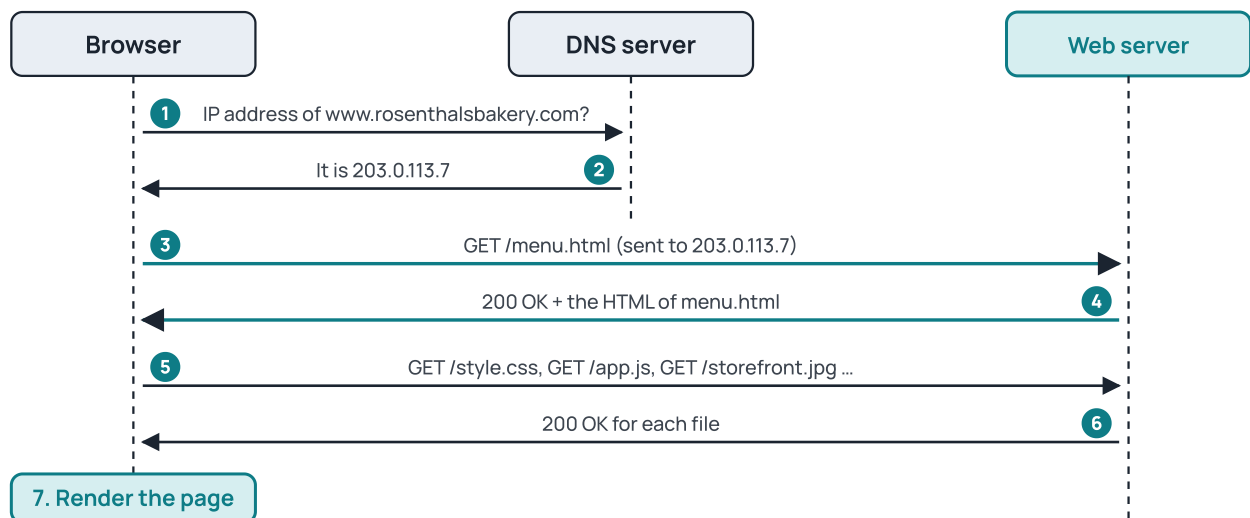
```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 5120

<!DOCTYPE html>
<html> ... the actual page ...
```

The first line carries a [status code](#), a three-digit summary of how things went. A handful are worth knowing because they show up in everyday life:

CODE	MEANING	WHEN YOU SEE IT
200 OK	Success, here is what you asked for	Every normal page load
301 Moved Permanently	That moved; go here instead	Old links that still work after a redesign
401 Unauthorized / 403 Forbidden	You need to log in / you're not allowed	Trying to open someone else's account page
404 Not Found	No such page	A typo in the address, a deleted page
500 Internal Server Error	The server's own code crashed	A bug in the site; the visitor did nothing wrong
503 Service Unavailable	The server is overloaded or down for maintenance	Ticket sales opening, launch-day traffic

One page usually triggers many requests. The HTML arrives first; as the browser reads it, it discovers it also needs `style.css`, `app.js`, and `storefront.jpg`, and sends a separate request for each. A typical commercial home page today makes 50 to 150 requests. This is why "the number of requests" is a real performance metric and why the CDN in Session 4 matters.



What happens after you press Enter. A name lookup (DNS) comes first, then one HTTP request for the page, then more requests for everything the page references. Time flows downward.

HTTPS: the padlock

Plain HTTP sends everything as readable text, so anyone on the same coffee-shop Wi-Fi could read the pages you request and anything you type into them.

HTTPS wraps the same conversation in **encryption**, so that only the browser and the server can read it. The padlock in the address bar means HTTPS is on. The server proves its identity with a *certificate* issued by a trusted authority; since 2015 these are free (from a non-profit called Let's Encrypt) and every serious hosting service turns them on automatically. There is no longer a business reason to run a site without HTTPS, and browsers actively warn visitors away from sites that do.

Getting the files onto a server: hosting

Hosting means renting space on someone else's always-on server and putting your files there. For a static website this is astonishingly cheap or free.

Services such as GitHub Pages, Netlify, Vercel, and Cloudflare Pages will take a folder of HTML, CSS, and JavaScript and serve it to the whole world, including HTTPS and a domain of your choosing, for nothing or a few dollars a month. The workflow is: put the files in the service, point your domain's DNS at it, done.

Why is it so cheap? Because serving static files is the easiest job a computer can do. The server reads a file from disk and sends it. No thinking, no computation per visitor, nothing to remember. That simplicity is the reason static sites are fast, hard to hack, and almost impossible to overload. It is also the reason they are limited, which brings us to the next section.

INSTRUCTOR NOTE

If time allows, open the browser's developer tools (F12 or right-click, Inspect) on any real site and show the Network tab. Students can watch the list of requests grow, see the status codes, and see the HTML arrive first. This makes the diagram above concrete in about 90 seconds.

2.6 Front end, back end, and the limits of static sites

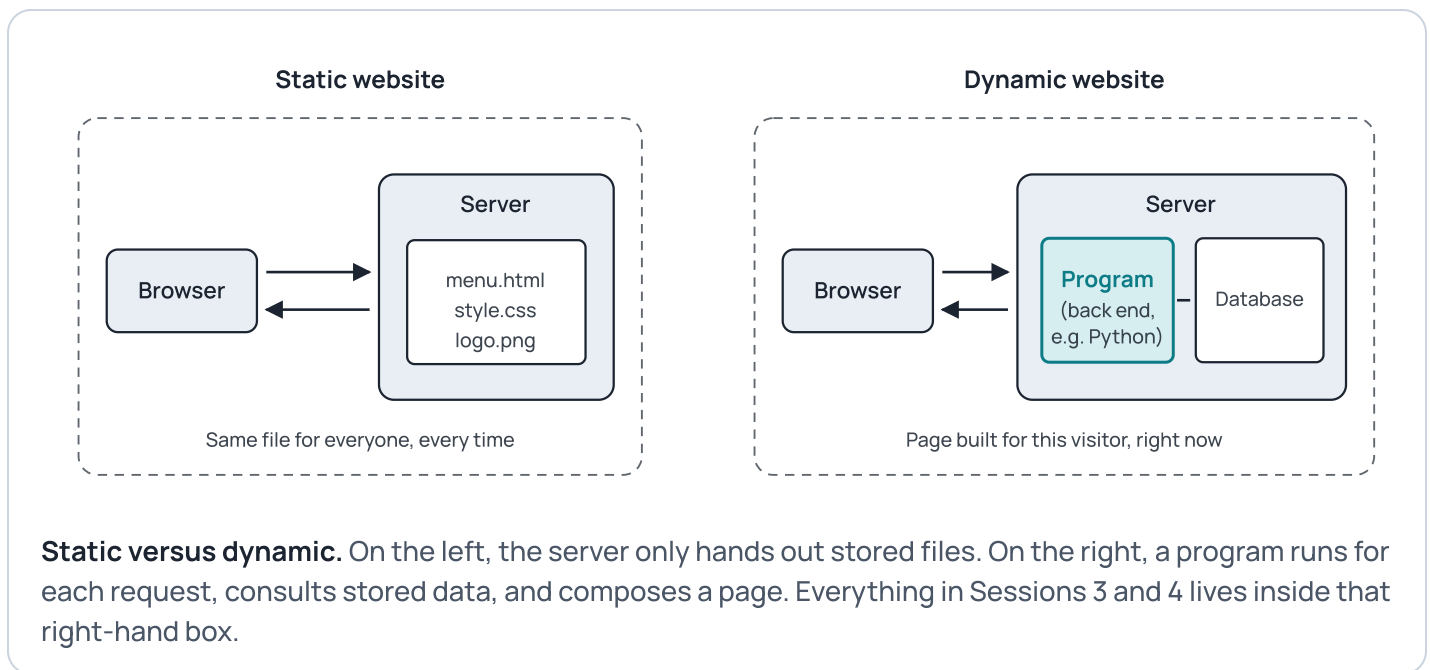
10 MIN

Two words you will hear in every conversation about software: **front end** and **back end**. The front end is everything that runs on the user's device and that they can see and touch: the HTML, CSS, and JavaScript we have been discussing. The back end is everything that runs on the server, out of sight: the code that decides what to send, checks who you are, records your order, charges your card.

A static website is front end only. That is precisely why it is cheap, and precisely why it cannot do certain things:

- **It cannot remember anything.** If a visitor fills in a form, there is nowhere for the data to go. The server just hands out files; it does not accept and store information.
- **It cannot personalise.** Every visitor gets the same `menu.html`. There is no "Welcome back, Dana" and no shopping cart that survives a page reload, because nothing on the server knows who Dana is.
- **It cannot keep secrets.** Everything in a static site is sent to the visitor's browser, so anything in it can be read by anyone. A password, a payment key, or a private price list cannot live in front-end code. This is one of the most important security rules in this module and it will return in Session 5.
- **It cannot do work on your behalf.** Sending a confirmation email, charging a credit card, or generating a PDF invoice all require a program running somewhere that is not the visitor's browser.

Every one of those needs is met by adding a back end: a program on the server that receives requests, runs logic, reads and writes stored data, and produces a response tailored to the visitor. A website with a back end is a **dynamic website**: its pages are generated at the moment they are requested, so they can be different for every person and every moment.



Most real products mix the two. The marketing pages of a company are static; the product behind the login button is dynamic. Knowing which parts of your product genuinely need a back end, and which can be static, is one of the first cost-saving architecture decisions a founder can make.

2.7 Where Python fits 5 MIN

You are learning Python in parallel with this course, so it is worth saying exactly where it fits into the picture. The browser can run only JavaScript. Python does not run in the browser. Python runs on *your* computer when you run a script, and it runs on *servers* as back-end code. When a Python program on a server answers web requests, it is the "Program" box in the diagram above.

Python is one of the most popular back-end languages in the world, used at Instagram, Spotify, Netflix, Dropbox, and nearly every AI company. In Session 3 you will see a complete, working Python web server in about ten lines. The Python skills from your other course are directly the skills of writing back ends.

LANGUAGE	WHERE IT RUNS	JOB
HTML, CSS	Browser	Content and appearance of pages
JavaScript	Browser (also servers via Node.js)	Interactivity in the page; also a common back-end language

LANGUAGE	WHERE IT RUNS	JOB
Python	Servers and your own computer	Back-end logic, data processing, AI, scripts and automation
SQL	Database server	Asking questions of stored data (Session 3)
Swift, Kotlin	Phones	Native iPhone and Android apps (Session 5)

Session 1 covered how a back-end language is chosen; the table above shows the result of that choice for this module. A final piece of vocabulary. A developer who works on the front end is a front-end developer; one who works on the back end is a back-end developer; one who does both is a [full-stack](#) developer. "The stack" is the list of technologies a product uses, top to bottom: the front-end framework, the back-end language, the database, the hosting. When someone asks "what's your stack?", they are asking about your architecture.

?

REVIEW QUESTIONS

Session 2: From a file to a page on the internet

1. In the client-server model, which side starts every conversation, and why does that matter for how servers are designed?
2. A web page is written in up to three languages. Name each one and state in one phrase what it is responsible for.
3. Why can you open a static website from a folder on your desktop with no internet connection, but your friend across town cannot see it?
4. Break the URL `https://shop.example.com/products/42?color=blue` into its parts and say what each part tells the browser.
5. What does DNS do, and what everyday object is it usually compared to?
6. A customer reports seeing "500 Internal Server Error" while another reports "404 Not Found." Which one suggests a bug in your own code, and which suggests a bad link or typo?
7. Give two things a purely static website cannot do, and explain in one sentence what has to be added to do them.
8. Your Python course teaches you to write programs. In a web application, where would that Python code run, and why not in the visitor's browser?

Answers are in [Appendix B, Session 2](#).

Dynamic Websites, APIs, and Databases

Session 2 ended with a program running on the server. This session opens that box. We see what a back end actually looks like, how the front end talks to it through an API, why almost every product needs a database, and what SQL is, illustrated with a bakery that finally takes orders.

BY THE END OF THIS SESSION STUDENTS CAN

- Read a ten-line Python web server and say what each route does.
- Explain what an API is, what JSON looks like, and what GET and POST mean.
- Describe the relational model: tables, rows, columns, primary and foreign keys.
- Read simple SQL statements and say in plain English what they do.
- Draw the three-tier architecture and trace an order through it.

SECTION	MINUTES
<u>3.1 Inside the back end: a Python web server</u>	12
<u>3.2 APIs and JSON: how programs talk to programs</u>	15
<u>3.3 Why every product needs a database</u>	10
<u>3.4 SQL by example</u>	15
<u>3.5 Beyond tables: NoSQL and ORMs</u>	8
<u>3.6 Putting it together: the three-tier architecture</u>	10
Review questions and buffer	5

3.1 Inside the back end: a Python web server 12 MIN

Here is a complete dynamic web server written in Python, using a popular library called Flask. It is short enough to read in full.

```

from flask import Flask
from datetime import datetime

app = Flask(__name__)

@app.route("/")
def home():
    return "<h1>Rosenthal's Bakery</h1><p>Fresh bread every morning.</p>"

@app.route("/hours")
def hours():
    now = datetime.now()
    if 7 ≤ now.hour < 18:
        return "<p>We are <b>open</b> right now!</p>"
    return "<p>We are closed. Back at 7 a.m.</p>"

app.run(port=5000)

```

Walk through it slowly with the class:

- `app = Flask(__name__)` creates the web server program. Flask is a **framework**: a bundle of pre-written code that handles the routine parts of being a web server, so the developer only writes the parts unique to the bakery. Flask, Django, and FastAPI are Python frameworks; Express and Next.js are JavaScript ones; Rails is Ruby; Spring is Java. When Claude Code builds you a web app, it will pick one of these.
- Each `@app.route("...")` line is a **route**: it connects a path (the part of the URL after the domain) to a Python function. When a request for `/hours` arrives, the `hours()` function runs and whatever it returns is sent back as the response.
- The `hours()` function is where "dynamic" happens. It checks the current time and returns different HTML depending on the answer. Two visitors a minute apart at 5:59 p.m. and 6:00 p.m. see different pages. No static file could do that.
- `app.run(port=5000)` starts listening. A **port** is a numbered door on a computer. One machine can run many server programs, each behind a different door. Websites use port 443 (HTTPS) and 80 (HTTP) by convention, so you never type them; developers testing on their own machine use doors like 5000 or 3000, which is why you will see addresses like `http://localhost:5000`.

localhost means "this computer." When you run the program above and open `http://localhost:5000/hours`, your browser is the client, your own laptop is the server, and the request never leaves the machine. Every developer works this way before putting anything on the internet. Session 4 covers the jump from localhost to the world.

LIVE DEMO (4 MINUTES)

1. Ask Claude Code to create the Flask app above and run it.
2. Open `http://localhost:5000` and `http://localhost:5000/hours` in a browser.
3. Change the closing hour in the code, save, refresh. Point out that the HTML did not exist as a file anywhere; the program composed it on demand.
4. Try `http://localhost:5000/nothing` to get a live 404.

In practice, back ends do not build HTML by gluing strings together. They use **templates**: HTML files with blanks in them, such as `Welcome back, {{ customer_name }}`, which the program fills in for each request. Designers can edit the template without touching the Python, and the Python can change without touching the design. The idea to carry away is the same either way: the server runs code, and the code produces the page.

3.2 APIs and JSON: how programs talk to programs

15 MIN

The Flask example returned HTML for a human to read. But increasingly the thing making the request is not a human's browser rendering a page. It is another program: JavaScript in the page asking for "just the data," a mobile app, a partner's system, or an AI assistant. For those callers, the server returns raw data instead of a designed page. The set of requests a server understands, and the shape of the answers it gives, is its **API** (Application Programming Interface).

ANALOGY

Back to the restaurant. The *menu* is the API. It lists exactly what you may order and what you will get. You do not need to see the kitchen, and the kitchen can be reorganised without changing the menu. A menu written in a standard format lets anyone order, including delivery apps that were never imagined when the restaurant opened.

A web API is reached over HTTP just like a page, but its **endpoints** (individual addresses) return data rather than HTML. Here is the bakery adding one:

```
@app.route("/api/products")
def list_products():
    return {
        "products": [
            {"id": 1, "name": "Sourdough loaf", "price": 7.50, "in_stock": True},
            {"id": 2, "name": "Rye bread", "price": 6.00, "in_stock": False},
            {"id": 3, "name": "Challah", "price": 8.00, "in_stock": True}
        ]
    }
```

A request to `/api/products` gets back this:

```
{
  "products": [
    {"id": 1, "name": "Sourdough loaf", "price": 7.5, "in_stock": true},
    {"id": 2, "name": "Rye bread", "price": 6.0, "in_stock": false},
    {"id": 3, "name": "Challah", "price": 8.0, "in_stock": true}
  ]
}
```

This format is **JSON** (JavaScript Object Notation, pronounced "Jason"). Curly braces hold a set of named values; square brackets hold a list; text is in quotes; numbers and true/false stand alone. Python students will recognise it: a JSON object is almost exactly a Python dictionary, and a JSON array is a Python list. Every programming language can read and write JSON, which is why it became the common tongue of the web.

Verbs: GET, POST, and friends

HTTP requests carry a **method**, a verb stating the kind of action. Static sites only ever needed GET ("give me this"). APIs use the full set:

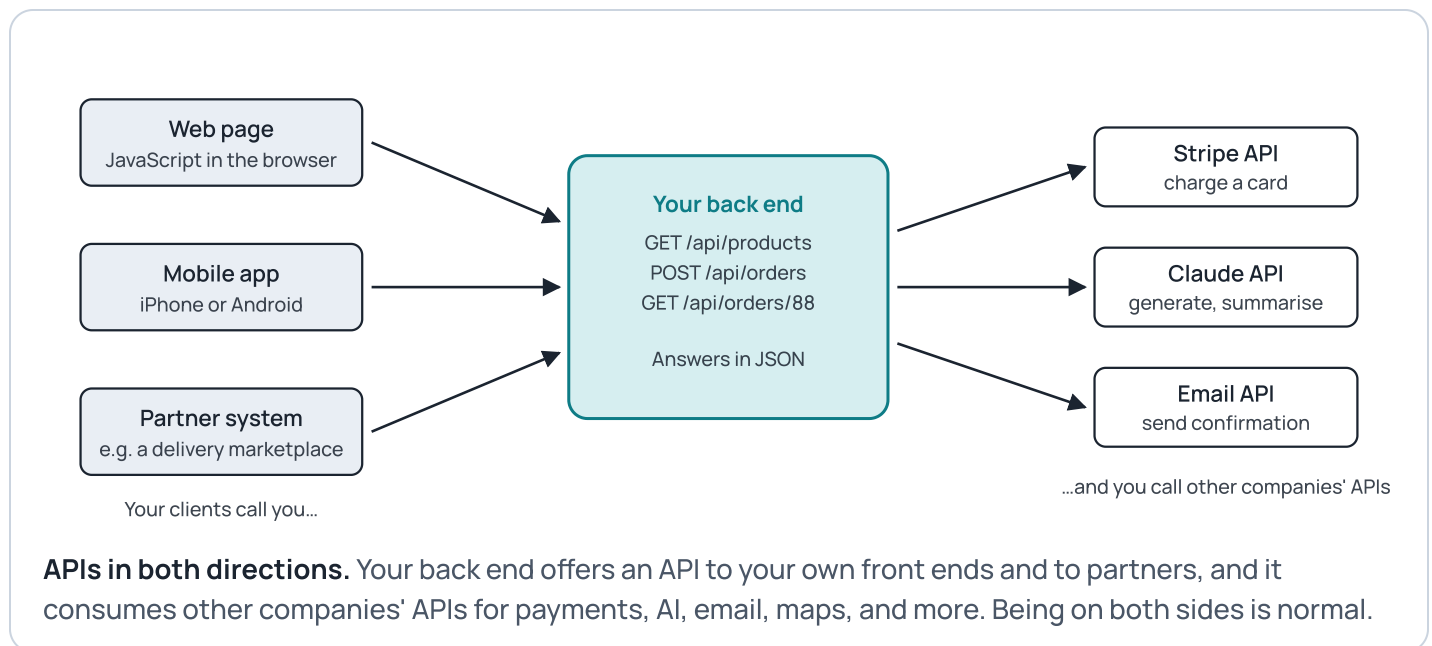
METHOD	MEANING	BAKERY EXAMPLE	BUSINESS ACTION
GET	Read; fetch something	GET <code>/api/products</code>	Show the catalogue
POST	Create; submit new data	POST <code>/api/orders</code> with the order in the body	Place an order
PUT / PATCH	Update something existing	PATCH <code>/api/orders/88</code> with <code>{"status": "ready"}</code>	Mark an order ready

METHOD	MEANING	BAKERY EXAMPLE	BUSINESS ACTION
DELETE	Remove	DELETE /api/orders/88	Cancel an order

An API organised this way, where every URL names a *thing* (a product, an order) and the method says what to do to it, is called **REST** (or "RESTful"). Almost every API you will encounter, from Stripe to Shopify to the Claude API, is roughly REST. The letters stand for something (Representational State Transfer) that nobody needs to remember; the idea is *nouns in the URL, verbs in the method*.

The four verbs map to the four things any system does with data, remembered by the acronym **CRUD**: Create, Read, Update, Delete. A surprising fraction of all business software is CRUD screens over a database: customers, invoices, tickets, inventory. Recognising a product as "mostly CRUD" is a useful signal that it is well understood and inexpensive to build.

Who calls APIs?



The right-hand side of this picture is where much of modern product-building happens. A **third-party API** lets you rent a capability instead of building it: charge a card with Stripe, send a text message with Twilio, look up an address with Google Maps, or generate text with the Claude API. Each is usually billed per use, which turns what used to be an engineering project into a line item. Much of "using AI to build products" is, architecturally, calling the right third-party APIs from your back end.

INSTRUCTOR NOTE

Students will meet the phrase "API key" constantly. It is a long secret string a third party gives you that identifies your account when your back end calls their API, so they know whom to bill. It lives on your server, never in front-end code. We return to this in Session 5, but planting it here pays off.

3.3 Why every product needs a database 10 MIN

The `/api/products` example had its data typed directly into the program. That breaks the moment a real business touches it: the baker cannot edit Python to change a price, and an order placed by a customer has nowhere to be saved. The server needs somewhere to *store* data that survives restarts, can be changed by the program, and can be searched. That somewhere is a database.

Why not just save to a file, the way you might save a spreadsheet? Because a database solves problems a file cannot:

- **Many writers at once.** Two customers buying the last challah at the same instant must not both succeed. A database handles this; a file being edited by two programs simultaneously gets corrupted.
- **Speed at scale.** Finding one order among ten million by scanning a file takes minutes. A database with an *index* (like the index in a textbook) does it in milliseconds.
- **Reliability.** If the power fails halfway through saving an order, a database guarantees the order is either fully saved or not saved at all, never half. This property is called a transaction. Moving money between two bank accounts is the classic case: the debit and the credit must both happen or neither.
- **Questions.** "Which customers ordered rye more than three times last quarter?" is one line in a database language and a small programming project against a pile of files.

The database is a separate program, a database management system (DBMS). Your Python back end does not store data itself; it sends instructions to the DBMS and gets results back, much as the browser sends requests to the web server. In Session 4 we will see that the DBMS often runs on its own machine.

The relational model: tables

The dominant way to organise business data for the last fifty years is the **relational model**. Data lives in **tables**: each table holds one kind of thing, each **row** is one instance of that thing, and each **column** is one fact about it. The bakery needs at least these:

customers

customer_id	name	email
1	Dana Levi	dana@example.com
2	Sam Ortiz	sam@example.com
3	Priya Nair	priya@example.co

products

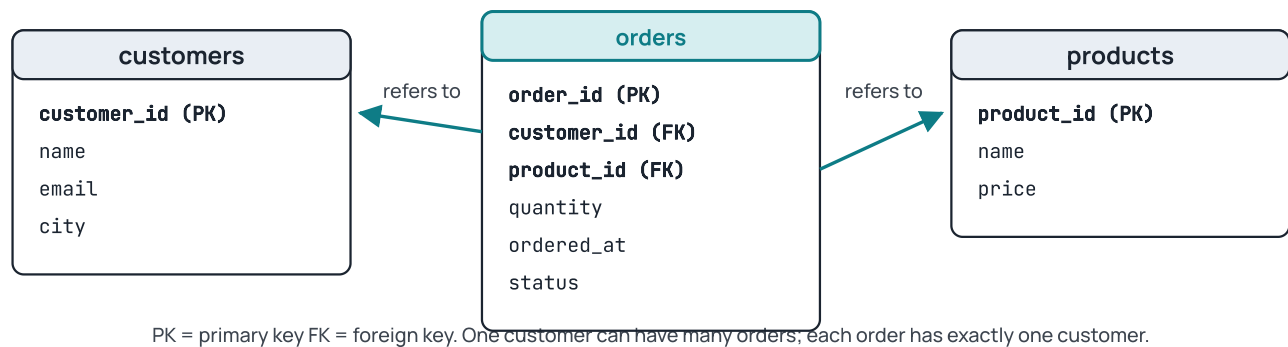
product_id	name	price
1	Sourdough loaf	7.50
2	Rye bread	6.00
3	Challah	8.00

orders

order_id	customer_id	product_id	quantity	ordered_at	status
87	1	3	2	2026-09-21 08:14	ready
88	2	1	1	2026-09-21 08:20	baking
89	1	2	1	2026-09-22 07:55	placed

Two ideas make this work:

- Every row has a **primary key**, a column whose value is unique for that row and never changes: `customer_id`, `product_id`, `order_id`. Names are not good keys because two customers can share one and people change theirs.
- Instead of copying Dana's name and email into every order she places, the order stores only her `customer_id`. A column that holds the primary key of a row in *another* table is a **foreign key**. The orders table has two: `customer_id` and `product_id`. This is what "relational" refers to: tables related through keys.



A schema for the bakery. Orders do not copy customer or product details; they point at them by key. Change Dana's email once and every order she ever placed reflects it.

The overall design of the tables, their columns, and the keys that link them is the **schema**. Designing a schema is really designing the business: what things exist, what we record about them, and how they relate. Business students are often better at this than engineers, because it is a question about the domain rather than about code.

3.4 SQL by example 15 MIN

Relational databases are all controlled with one language, **SQL** (Structured Query Language, said "sequel" or "S-Q-L"). It was designed in the 1970s to be readable by business analysts, and it largely is. You are not expected to write SQL in this course; you are expected to recognise it, read simple statements, and know what it can do. Claude Code will write the SQL, but you will be reviewing what it writes and asking for the right things.

A request to a database is a **query**. Six examples cover most of what a business ever needs.

Reading: SELECT

```
SELECT name, email FROM customers WHERE city = 'Boston';
```

"Give me the name and email of every customer whose city is Boston." **SELECT** picks the columns, **FROM** picks the table, **WHERE** filters the rows. Result: Dana Levi and Priya Nair. Replace the column list with `*` to get every column.

Counting and summing: aggregate functions

```
SELECT status, COUNT(*) AS how_many
FROM orders
GROUP BY status;
```

"For each order status, how many orders are there?" Result: one row per status (placed 1, baking 1, ready 1). `SUM`, `AVG`, `MIN`, and `MAX` work the same way. Every management dashboard in the world is a stack of queries like this.

Combining tables: JOIN

```
SELECT customers.name, products.name, orders.quantity
FROM orders
JOIN customers ON orders.customer_id = customers.customer_id
JOIN products  ON orders.product_id  = products.product_id
WHERE orders.status = 'placed';
```

This is the query that makes the relational model pay off. The orders table has only ID numbers; a `JOIN` stitches each order to the matching customer row and product row using the keys, so the result reads like a human wants it to: "Dana Levi, Rye bread, 1." Whenever you see a report that combines information from several places, there is a JOIN behind it.

Writing: INSERT, UPDATE, DELETE

```
INSERT INTO orders (customer_id, product_id, quantity, status)
VALUES (3, 1, 2, 'placed');

UPDATE orders SET status = 'ready' WHERE order_id = 88;

DELETE FROM orders WHERE order_id = 89;
```

"Add an order: customer 3, product 1, quantity 2, status placed." "Change order 88's status to ready." "Remove order 89." The database assigns the new order its `order_id` automatically. Notice how much rides on the `WHERE` clause: `DELETE FROM orders;` with no `WHERE` deletes every order in the company. This is not a hypothetical. It is a well-known category of career-defining mistakes, and the reason production databases have backups and restricted permissions (Session 4).

Creating the structure: CREATE TABLE

```
CREATE TABLE orders (
  order_id      INTEGER PRIMARY KEY,
  customer_id   INTEGER REFERENCES customers(customer_id),
  product_id    INTEGER REFERENCES products(product_id),
  quantity      INTEGER NOT NULL,
  ordered_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  status        TEXT DEFAULT 'placed'
);
```

This defines the table itself: each column, its data type (whole number, timestamp, text), which column is the primary key, which columns are foreign keys (`REFERENCES`), and which are required (`NOT NULL`). The database will now refuse to save an order with no quantity, or one pointing at a customer who does not exist. Those constraints are business rules enforced by the storage layer, which is far more reliable than hoping every program remembers to check.

WHAT STUDENTS SHOULD TAKE AWAY

- SQL is a language of questions and instructions about tables. It reads roughly like English.
- Reading is `SELECT` ; writing is `INSERT` , `UPDATE` , `DELETE` ; structure is `CREATE TABLE` .
- `JOIN` is how information split across tables is put back together.
- They should be able to look at a query Claude Code writes and say "this deletes every row" or "this only fetches Boston customers."

Which database? The names you will hear

PRODUCT	TYPE	TYPICAL USE	COST MODEL
SQLite	Relational, single file	Prototypes, phone apps, small tools. Built into Python. Claude Code will often start you here.	Free, no server to run
PostgreSQL ("Postgres")	Relational server	The default choice for new web products of any size	Free software; hosted versions charged by size and usage
MySQL / MariaDB	Relational server	Very common in older web products, WordPress, many hosting packages	Free software; hosted versions charged

PRODUCT	TYPE	TYPICAL USE	COST MODEL
Microsoft SQL Server, Oracle	Relational server	Large enterprises, finance, legacy systems	Commercial licences, often expensive
MongoDB	Document (NoSQL)	Flexible or rapidly changing data shapes	Free software; hosted service (Atlas) charged
Redis	Key-value, in memory	Caching, sessions, counters (Session 4)	Free software; hosted charged

All the relational ones speak SQL with minor dialect differences, so the concepts transfer. For a new product in 2026, "Postgres" is the safe default and the one you will hear most.

3.5 Beyond tables: NoSQL and ORMs 8 MIN

NoSQL databases

Not all data fits neatly into fixed columns. A product catalogue where every product has different attributes (shoes have sizes, laptops have memory) fights the relational model. [NoSQL](#) databases relax the rules. The most common kind, a *document database* such as MongoDB, stores each record as a JSON-like document with whatever fields it needs:

```
{
  "product_id": 3,
  "name": "Challah",
  "price": 8.00,
  "tags": ["bread", "shabbat", "egg"],
  "sizes": {"small": 6.00, "large": 8.00}
}
```

Flexibility has a price: the database no longer enforces that every product has a price, and combining data across collections is clumsier than a JOIN. The choice between relational and NoSQL is a genuine architecture decision, but a simple rule of thumb serves most businesses: **start relational**. Business data (customers, orders, invoices, inventory) is naturally tabular and you will want the guarantees. Reach for NoSQL when you have a specific reason: extremely high write volume, genuinely irregular data, or a cache.

ORMs: letting Python speak to the database

Developers rarely type SQL into their programs directly. Most use an **ORM** (Object-Relational Mapper), a library that lets the program treat rows as ordinary Python objects and generates the SQL behind the scenes:

```
# Python with an ORM (SQLAlchemy):
boston = Customer.query.filter_by(city="Boston").all()

# ...which the ORM turns into SQL for you:
# SELECT * FROM customers WHERE city = 'Boston';
```

When Claude Code builds a back end for you, it will almost certainly use an ORM, and files called "models" describe the tables. The point for a business reader: the concepts (tables, keys, queries) are unchanged; the ORM is a convenience layer on top of them.

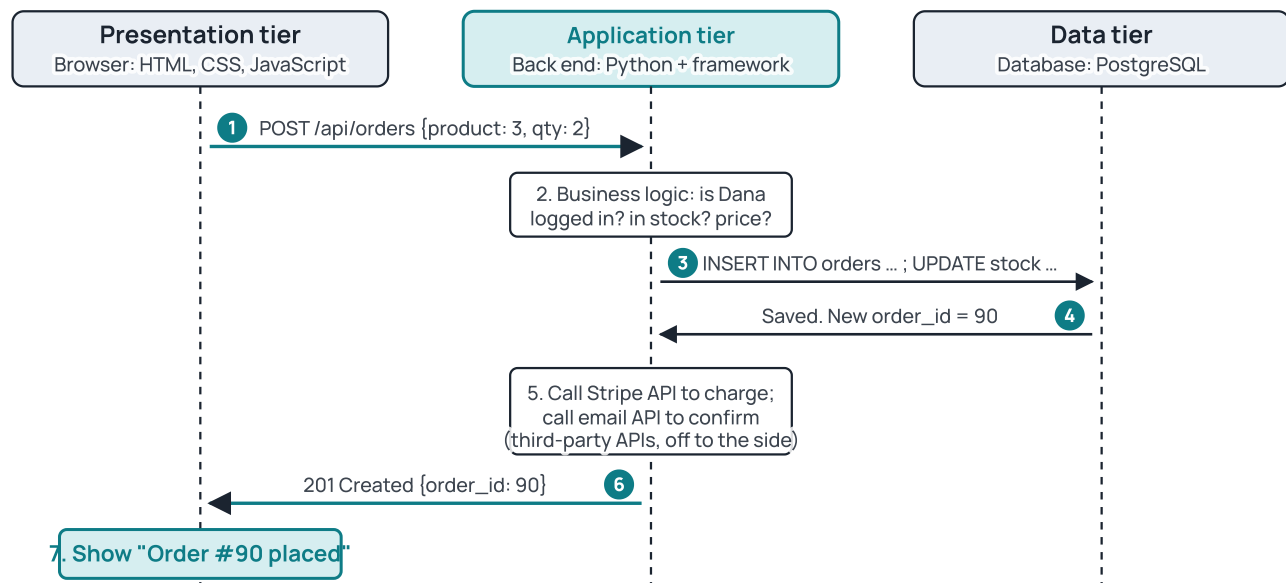
3.6 Putting it together: the three-tier architecture

10 MIN

We now have three distinct kinds of component, and the standard name for arranging them is the **three-tier architecture**:

1. **Presentation tier**: the front end. HTML, CSS, and JavaScript in the browser (or the screens of a phone app). Shows things to people and collects their input.
2. **Application tier** (also called the logic tier or the back end): the server program. Receives requests, enforces business rules, decides what to do, talks to the database, and returns responses. This is where **business logic** lives: prices, discounts, permissions, validation, workflows.
3. **Data tier**: the database. Stores and retrieves the facts.

Follow one order through all three tiers. Dana clicks "Order 2 challahs":



One order through three tiers. The browser never touches the database and never sees the payment key. Everything that must be trusted happens in the application tier, on the server.

Three observations that will matter for the rest of the module:

- **The browser never talks to the database directly.** All access goes through the back end, which checks identity and applies rules. If a browser could send SQL, anyone could run `DELETE FROM orders`. This is why the front end can never be trusted with secrets or with enforcing rules; anything in the browser can be changed by the person using it.
- **Each tier can be changed independently.** Redesign the front end without touching the database. Swap SQLite for PostgreSQL without changing the HTML. Rewrite the back end from Python to something else while the other two tiers carry on. This independence is the whole point of tiers.
- **Each tier can be placed on different hardware.** Today all three might run on one laptop. Tomorrow the database can move to its own machine and the back end can be copied onto ten. That is Session 4.

INSTRUCTOR NOTE

Ask students to name the three tiers for an app they use daily. For Uber: the app on the phone (presentation), Uber's servers matching riders and drivers and computing fares (application), and the records of trips, drivers, and payments (data). Then ask where the surge-pricing rule lives. It is in the application tier, on the server, precisely so that no one can tamper with it from the app.

Session 3: Dynamic websites, APIs, and databases

1. In the Flask example, what does a route do, and what is the difference between the `/` route and the `/hours` route from the visitor's point of view?
2. Explain what an API is using the restaurant analogy, then give a real example of a third-party API a small business might call and why.
3. What is JSON, and why did it become the standard format for APIs?
4. Match each HTTP method to a CRUD operation and give a bakery example for each.
5. Give three reasons a business stores its data in a database rather than in ordinary files.
6. In the bakery schema, why does the orders table store a `customer_id` instead of the customer's name and email? Which term describes that column?
7. In plain English, what does this query return?

```
SELECT customers.name, COUNT(*) FROM orders JOIN customers ON orders.customer_id = customers.customer_id GROUP BY customers.name;
```
8. What is dangerous about the statement `DELETE FROM orders;` and what single addition would make it safe?
9. Name the three tiers of the three-tier architecture and state which tier enforces the rule "customers get 10% off orders over \$50" and why it must be there.

Answers are in [Appendix B, Session 3](#).

Hosting, Scaling, and the Cloud

So far everything has run on one laptop. This session moves the bakery onto the internet and then grows it, one step at a time, from a single rented server to the distributed architecture used by companies serving millions of people. Each step is motivated by a specific problem the previous step could not solve, and each step has a price tag.

BY THE END OF THIS SESSION STUDENTS CAN

- Explain what deployment means and name the standard environments code moves through.
- Describe the single-server setup and its main weakness.
- Explain why the database is moved to its own machine.
- Describe horizontal scaling, load balancing, and why servers must be stateless.
- Say what a cache, a CDN, and object storage each do.
- Distinguish IaaS, PaaS, SaaS, and serverless, and know when each is a sensible choice.

SECTION	MINUTES
<u>4.1 From localhost to production</u>	10
<u>4.2 Step 1: everything on one server</u>	10
<u>4.3 Step 2: give the database its own machine</u>	10
<u>4.4 Step 3: many web servers behind a load balancer</u>	15
<u>4.5 Step 4: caches, CDNs, and storage</u>	10
<u>4.6 The cloud menu: IaaS, PaaS, SaaS, serverless</u>	10
<u>4.7 Shipping code safely</u>	5
Review questions and buffer	5

4.1 From localhost to production 10 MIN

The bakery's back end works beautifully at `http://localhost:5000`. Moving it to a place where customers can reach it is called **deployment**. "Deploy" is one of the most frequently used verbs in a software team, and it means exactly this: take the code that works here and make it run there.

Serious teams do not deploy straight from a laptop to customers. Code passes through **environments**, each a full copy of the application with its own database:

ENVIRONMENT	WHO USES IT	PURPOSE	DATA
Development ("dev", "local")	Each programmer, on their own computer	Write and try code freely; break things	Fake or sample data
Staging ("test", "QA")	The team, testers, sometimes the client	A private replica of production to check a release before customers see it	Realistic but not real (or anonymised)
Production ("prod", "live")	Real customers	The actual business	Real; protected and backed up

"It works on my machine" is the oldest joke in software precisely because development and production differ: different operating system, different database size, different traffic, different secrets. Staging exists to catch those differences before a customer does. When a founder asks "can we just push it live?" the honest engineering answer is "only if you would be comfortable finding the bugs at the same moment your customers do."

Two measures describe how well a production system is doing. **Uptime** is the percentage of time it is reachable; each extra "nine" is dramatically more expensive to achieve. **Latency** is how long each request takes to answer. Much of the architecture in this session exists to push uptime up and latency down.

UPTIME	DOWNTIME ALLOWED PER YEAR	TYPICAL OF
99%	About 3.7 days	A hobby site on one cheap server

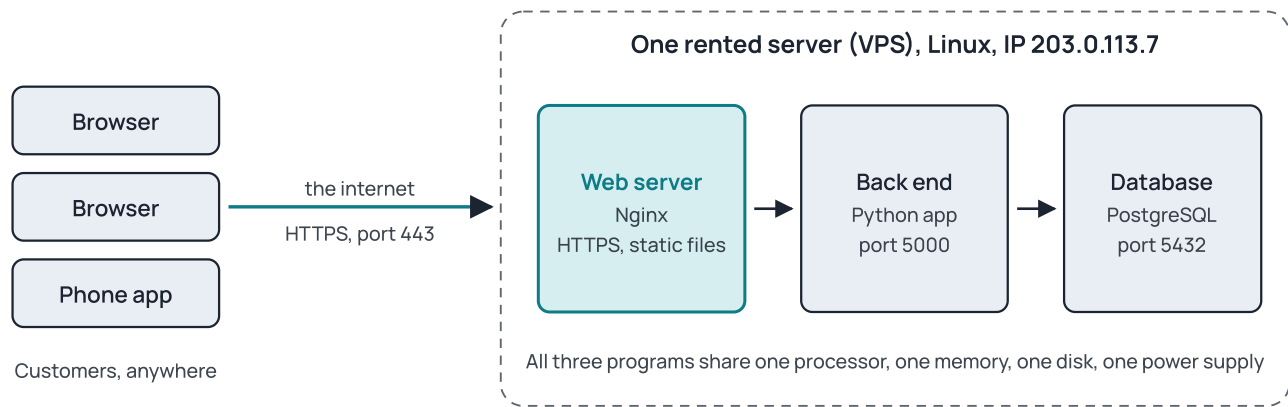
UPTIME	DOWNTIME ALLOWED PER YEAR	TYPICAL OF
99.9% ("three nines")	About 8.8 hours	A small business product with sensible hosting
99.99% ("four nines")	About 53 minutes	Payment, banking, large SaaS. Requires redundancy at every layer
99.999% ("five nines")	About 5 minutes	Telecom, critical infrastructure. Very expensive

4.2 Step 1: everything on one server 10 MIN

The simplest deployment is to rent one computer in a data center and install everything on it. Cloud companies rent these by the hour or month as [virtual private servers](#) (VPS) from providers such as DigitalOcean, Linode, Hetzner, or Amazon's EC2, for as little as 5 dollars a month. "Virtual" because one powerful physical machine is carved into several independent virtual ones, each of which feels to its renter like a whole computer.

On that one machine you install:

- An [operating system](#), almost always Linux on servers. Linux is free, stable, and what nearly all server software is built for. Windows and macOS run desktops; iOS and Android run phones; Linux runs the internet.
- The web server program (Nginx is the usual choice), which accepts connections on ports 80 and 443, handles HTTPS, serves the static files itself, and forwards dynamic requests to the back end.
- Your back-end program (the Python/Flask app), listening on an internal port such as 5000.
- The database (PostgreSQL), listening on its own internal port, 5432 by convention.



Step 1: the single-server deployment. Simple, cheap, and enough for thousands of daily visitors. Every component shares the same hardware, so every component shares the same fate.

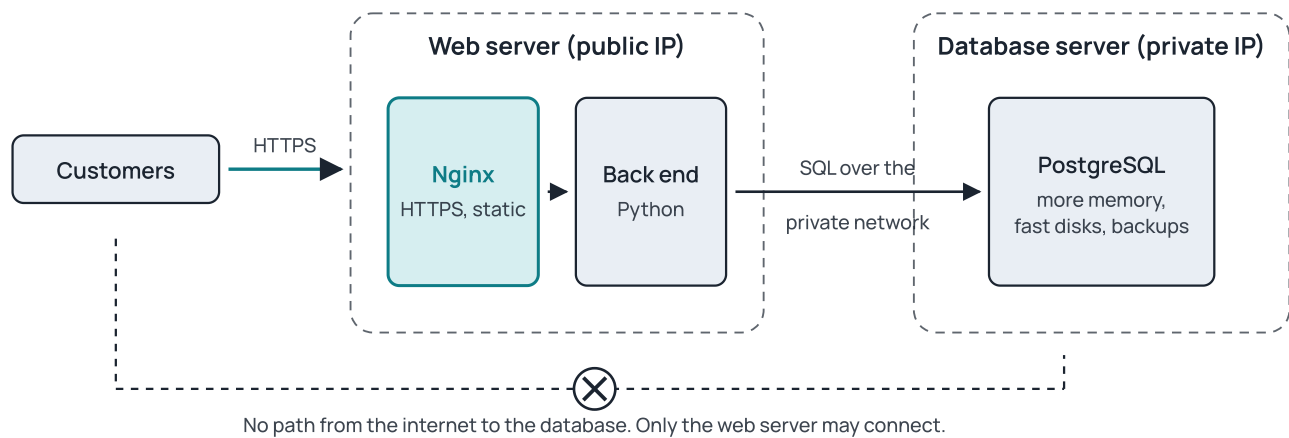
Do not underestimate this setup. A modest 20-dollar server can comfortably handle tens of thousands of visits a day, which covers most small businesses and every early-stage prototype. Its weaknesses only appear with growth:

- **Single point of failure.** If the machine's disk dies, or the data center loses power, or you make a mistake while updating it, the website and the database disappear together. A **single point of failure** is the enemy of uptime.
- **Resource contention.** The database and the back end compete for the same memory and processor. A heavy report query can make the website slow for every visitor.
- **Hard ceiling.** When traffic outgrows the machine, the only option is a bigger machine, and machines do not get infinitely big.
- **Security blast radius.** If an attacker breaks into the web server, they are already sitting on the same machine as all the customer data.

Each of the following steps removes one of these weaknesses, and each adds cost and complexity. Good architecture means taking the next step when the problem is real, not before.

4.3 Step 2: give the database its own machine 10 MIN

The first thing to move off the shared server is the database. It goes onto a second machine, a dedicated **database server**, and the back end connects to it over the network instead of talking to it on the same box.



Step 2: the database moves to its own machine on a private network. The public internet can reach the web server but not the database. Each machine can now be sized for its own job.

Why this is worth the second bill:

- **Right-sized hardware.** Databases want lots of memory and fast disks; web servers want processor cores and network bandwidth. Separated, each can be bought for what it actually needs.
- **Security.** The database server gets a private network address only. There is no route from the internet to it. A firewall allows connections on port 5432 only from the web server. An attacker who compromises the web server still has another wall to get through, and no one can stumble across the customer records from outside.
- **Independent failure and maintenance.** The web server can be rebuilt, upgraded, or replaced without touching a byte of data. Backups of the database run without slowing the website.
- **It unlocks the next step.** With the data in one shared place, you can now run more than one copy of the web server, which is what Step 3 does.

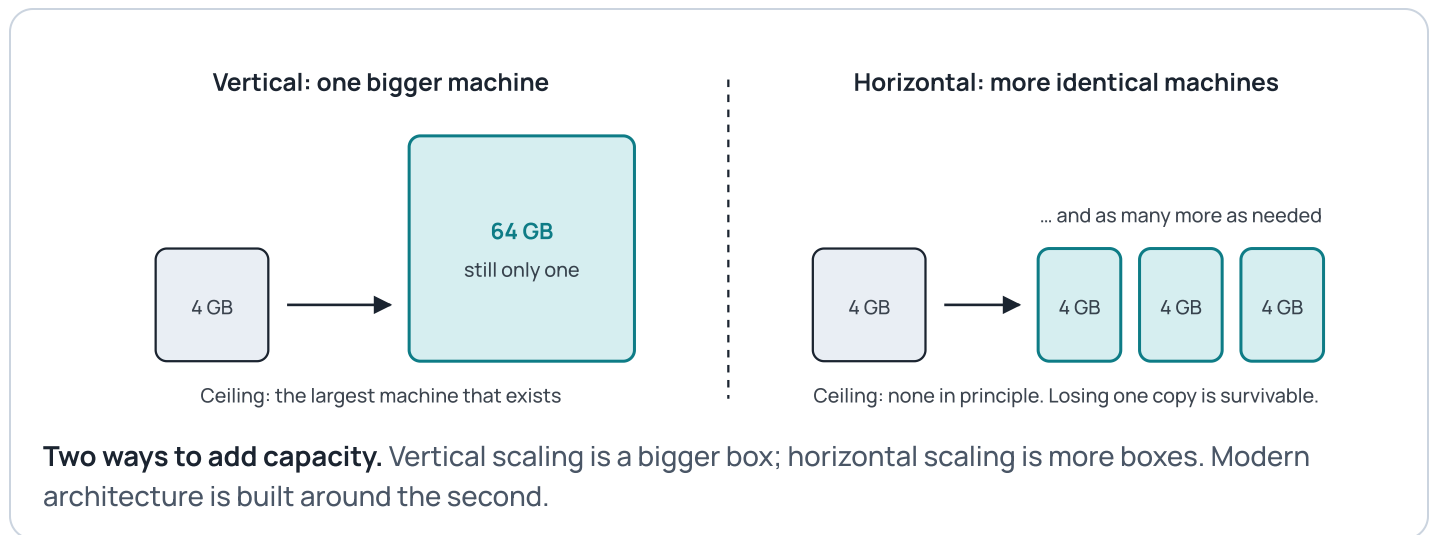
In practice, almost nobody installs PostgreSQL on a rented machine themselves any more. Cloud providers offer a managed database, such as Amazon RDS, Google Cloud SQL, Supabase, or Neon: you click "create PostgreSQL," and they run the machine, take nightly backups, install security patches, and can switch to a standby copy automatically if the primary fails. You get a connection address and a bill, starting around 15 to 50 dollars a month. For nearly every business this is the correct choice; running your own database is expertise you do not want to have to hire.

4.4 Step 3: many web servers behind a load balancer

15 MIN

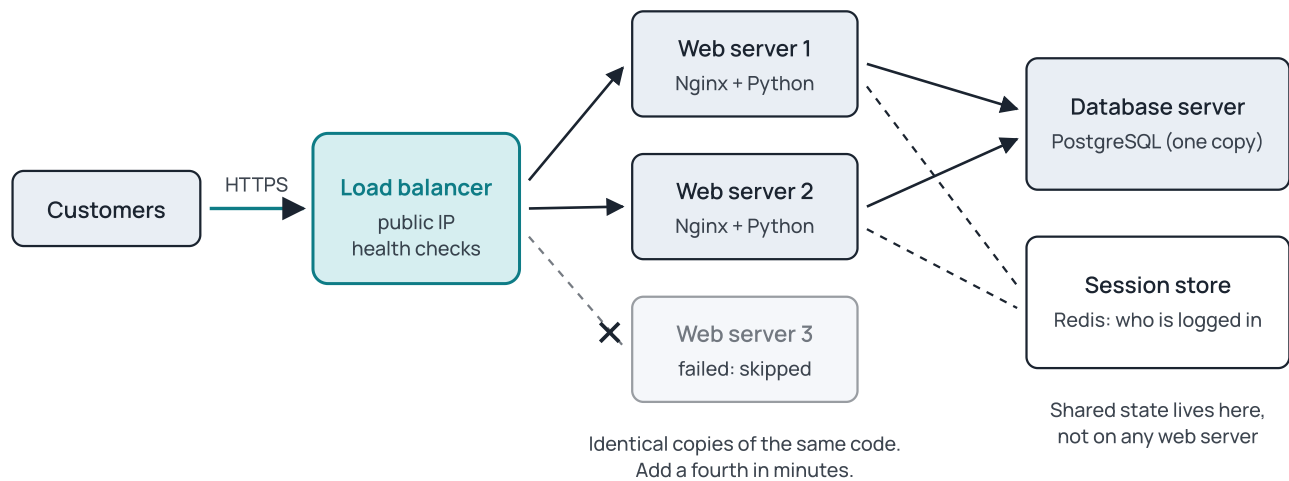
The bakery is featured on a national morning show. Ten thousand people open the site in the same minute. The single web server, already the busiest machine, runs out of capacity and starts answering with 503 errors. There are two ways to add capacity, and their names are worth knowing because they come up in every scaling conversation.

- **Vertical scaling** ("scaling up"): replace the machine with a bigger one. Simple, no code changes. But the biggest machine money can buy is still finite, price grows faster than capacity, and there is still exactly one of it.
- **Horizontal scaling** ("scaling out"): keep the machines modest but run several identical copies of the web server. Twice the traffic, twice the servers. In principle there is no ceiling; this is how every large internet company works.



The load balancer

Several web servers create an immediate question: the domain name resolves to *one* IP address, so which server does a customer's request go to? The answer is a new component that sits in front of all of them, the **load balancer**. The public address belongs to the load balancer. It accepts every request and forwards each to one of the servers behind it, spreading the work evenly (round-robin, or to whichever is least busy). It also performs **health checks**: every few seconds it pings each server, and if one stops answering it simply stops sending it traffic. Customers never notice.



Step 3: horizontal scaling. The load balancer is the only public address. Web servers are interchangeable and disposable; anything that must be remembered between requests lives in the shared database or session store.

The catch: servers must be stateless

Horizontal scaling has one requirement that shapes how back ends are written. Consider a customer who logs in. Their first request goes to web server 1, which notes "this person is Dana, logged in." Their next click is routed by the load balancer to web server 2, which has never heard of them and asks them to log in again. The site appears broken.

The solution is that web servers must be **stateless**: they remember nothing between requests. Everything that needs remembering (who is logged in, what is in the shopping cart) is stored somewhere all the servers can reach: the database, or a fast shared store such as Redis. Any server can then handle any request, because the information travels with the request (in a cookie, as Session 5 explains) and the facts live in the shared store. Statelessness is what makes servers interchangeable, and interchangeable servers are what make it possible to add ten more during the morning show and remove them after lunch.

This step is also the one that turns "the server is down" from a disaster into a non-event. With two or more servers behind a load balancer, one can fail, be updated, or be replaced while the others carry on. Redundancy at every layer, including the database, is what the industry calls **high availability**, and it is how three and four nines of uptime are bought.

BUSINESS TRANSLATION

Horizontal scaling is what makes cloud costs *elastic*: pay for eight servers during Black Friday and two in February. It is also why a founder should ask, before launch, "Can we run two copies of this?" If the answer is no, because the code keeps things in memory on one machine, the first traffic spike will expose it.

4.5 Step 4: caches, CDNs, and storage 10 MIN

With the servers scaled, the next bottlenecks are usually the database and the distance between servers and customers. Three components address them.

Cache

A [cache](#) is a fast, temporary copy of something that was expensive to produce. The bakery's product list is requested thousands of times a minute and changes twice a day. Running the same SQL query thousands of times is wasteful; the back end runs it once, keeps the result in memory (typically in Redis, the same in-memory store used for sessions), and hands out the copy for the next 60 seconds. Database load drops by a thousand-fold. Caches appear everywhere: your browser caches images so it does not re-download them, servers cache query results, and the CDN below is a cache too. Their one hard problem is *staleness*: the copy can be out of date, so someone must decide how long a copy may live and when to throw it away.

Content delivery network

A customer in Singapore requesting an image from a server in Virginia waits for the bytes to cross the Pacific twice, about 200 milliseconds, before anything appears. A [CDN](#) (Content Delivery Network) is a company with servers in hundreds of cities that keeps copies of your static files (images, CSS, JavaScript, video) near your users. The Singaporean customer gets the image from Singapore in 10 milliseconds, and your own servers never see the request at all. Cloudflare, Akamai, Fastly, and Amazon CloudFront are the household names. Most of the bytes of most websites are served by a CDN, not by the site's own servers, and CDNs also absorb the flood of junk traffic in a denial-of-service attack.

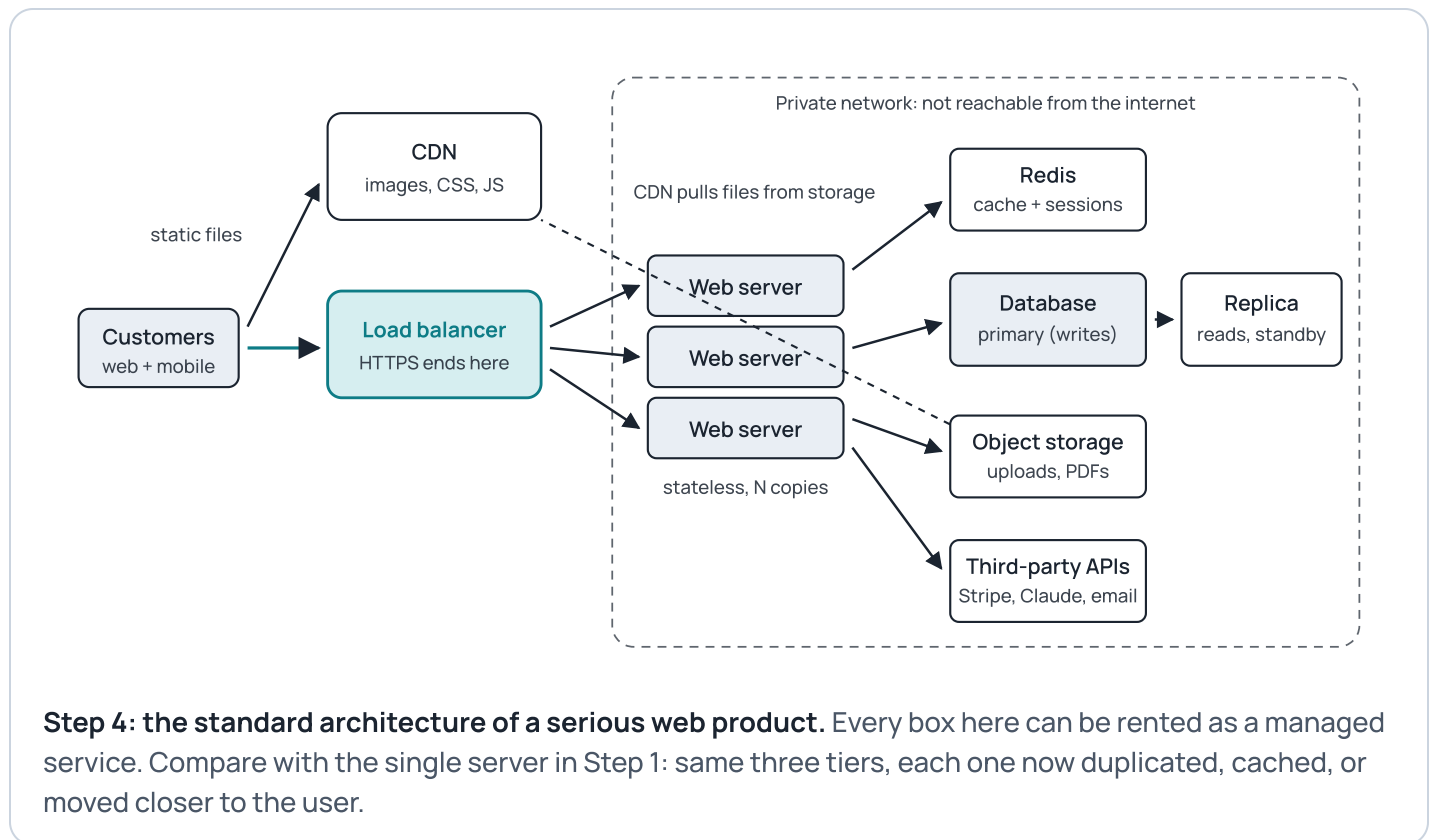
Object storage

Customers upload photos; the bakery publishes menus as PDFs; the app records videos. Files like these should not live on a web server's disk, because with three web servers a file uploaded to server 1 is invisible to servers 2 and 3. They go into [object storage](#), a cloud service designed to hold unlimited

files at fractions of a cent per gigabyte and hand them out by URL. Amazon S3 is the original and the name people use generically; Google Cloud Storage, Azure Blob Storage, and Cloudflare R2 are equivalents. The database stores the *address* of the photo; the photo itself is in object storage, usually behind the CDN.

Read replicas

Finally, the database itself can be partially scaled. Most applications read far more than they write. A **read replica** is a continuously updated copy of the primary database that answers read-only queries (product lists, order history, reports) while all writes still go to the one primary. A replica also serves as a hot spare: if the primary fails, the replica is promoted and the site carries on. Splitting writes across several databases (called sharding) is the hard, expensive step that only very large companies take; it is beyond this course, but you should know the word.



This picture, or something very like it, is what runs behind most products you use. Naming its parts and explaining why each exists is the core competence this module aims for. Notice that nothing conceptually new was added after Session 3: it is still presentation, application, and data. Everything in this session was about copies, placement, and failure.

4.6 The cloud menu: IaaS, PaaS, SaaS, serverless 10 MIN

Everything above can be rented rather than owned, and [cloud computing](#) is simply the name for that arrangement: someone else owns the data centers and you pay for what you use. Amazon Web Services (AWS), Microsoft Azure, and Google Cloud are the giants; there are many smaller, simpler providers. The real decision is not which brand but *how much of the work you want to do yourself*, and the industry describes that choice in layers.

LAYER	YOU GET	YOU MANAGE	EXAMPLES	GOOD FOR
IaaS Infrastructure as a Service	Virtual machines, disks, networks	Operating system, web server, database, security patches, scaling, backups	Amazon EC2, Google Compute Engine, DigitalOcean, Hetzner	Teams with operations expertise; unusual requirements
PaaS Platform as a Service	"Here is my code; run it"	Just your code and your data	Heroku, Render, Railway, Fly.io, Vercel, Netlify, Google App Engine	Startups, small teams, anything built with Claude Code
SaaS Software as a Service	A finished product	Your account settings	Gmail, Salesforce, Shopify, Slack, Notion, QuickBooks	Everything you do not need to build yourself
Serverless (Functions as a Service)	Run this function when something happens	Just the function's code	AWS Lambda, Cloudflare Workers, Vercel Functions, Google Cloud Functions	Spiky or infrequent workloads; glue between services

ANALOGY

Getting a pizza. IaaS is renting a kitchen: you bring ingredients and cook. PaaS is a meal kit: ingredients and instructions arrive, you assemble. SaaS is delivery: a finished pizza at the door. Serverless is a vending machine that bakes a slice when you press the button and charges you only for that slice. Each step to the right costs more per unit but less in effort and staff, and the right answer depends on how many pizzas you sell and whether pizza is your business.

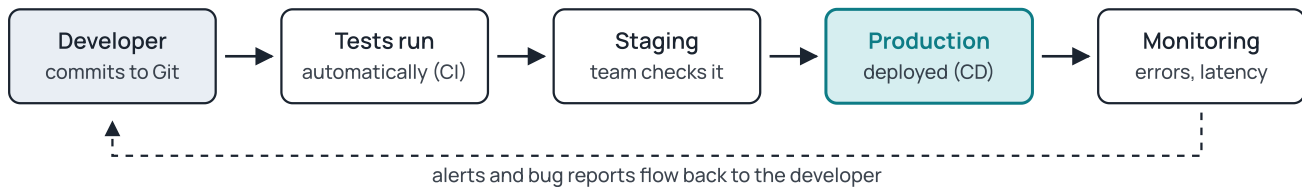
Two more terms fill out the vocabulary. A **container** (made with a tool called Docker) packages a program with everything it needs so it runs identically on a developer's laptop and on any cloud, which largely cures "it works on my machine." Kubernetes is the system that runs and scales thousands of containers across many machines; it is powerful, complex, and almost never the right first choice for a small team. When people say "we containerised the app," they mean it is now portable across all the layers above.

For students in this course, the practical guidance is direct. A prototype built with Claude Code is best deployed on a PaaS (Render, Railway, Vercel, Fly.io) with a managed database (Supabase, Neon, or the PaaS's own). That gives you Steps 1 through 3 of this session, including HTTPS, load balancing, and horizontal scaling, for somewhere between 0 and 50 dollars a month, without ever logging into a server. Move down to IaaS only when you have a specific reason and someone to operate it.

4.7 Shipping code safely 5 MIN

Three practices make it possible to change a live product without breaking it, and you will see all three in any team you work with.

- **Version control**. Every change to the code is recorded, with author and reason, and can be undone. The tool is Git; GitHub and GitLab host the shared copies. A *repository* ("repo") is one project's history. Claude Code works inside Git repositories and can commit changes for you; knowing that the history exists, and that anything can be rolled back, is what makes experimenting safe.
- **CI/CD** (Continuous Integration / Continuous Deployment). Automation that runs the tests every time code changes and, if they pass, deploys it. Instead of a risky quarterly release, teams ship small changes many times a day. On a PaaS this is often built in: connect your GitHub repo, and every change to the main branch goes live within minutes.
- **Monitoring** and logging. Servers write a line to a log for every request and every error; monitoring tools (Datadog, Sentry, Grafana) collect them, chart them, and page someone at 2 a.m. if the error rate jumps. A product without monitoring finds out about outages from angry customers.



How code reaches customers. Each arrow is automated. A change that fails its tests never reaches staging; one that misbehaves in production is caught by monitoring and rolled back through version control.

?

REVIEW QUESTIONS

Session 4: Hosting, scaling, and the cloud

1. What does "deploy" mean, and why do teams put a staging environment between development and production?
2. Describe the Step 1 single-server deployment and name its most serious weakness.
3. Give two reasons for moving the database onto its own server, and explain what "private network" means in that design.
4. Explain the difference between vertical and horizontal scaling. Which one does modern web architecture rely on, and why?
5. What does a load balancer do, and what is a health check?
6. Why must web servers be stateless in a load-balanced setup? Where does the "state" go instead?
7. A customer in Tokyo complains that images load slowly from your site hosted in Virginia. Which component fixes this and how?
8. What is a cache, and what is the one hard problem every cache has?
9. Place each of these on the IaaS / PaaS / SaaS / serverless menu: renting a virtual machine on AWS EC2; using Shopify to run a store; connecting a GitHub repo to Render so it runs your code; running a small function on AWS Lambda when a file is uploaded.
10. You are deploying a prototype built with Claude Code for a business school project. Which layer of the cloud menu is the sensible choice and why?

Answers are in [Appendix B, Session 4](#).

Identity, Security, and Software Beyond the Browser

Every architecture so far has quietly assumed the server knows who is asking. This session explains how it finds out: passwords, cookies and sessions, Basic Auth, and OAuth, the mechanism behind every "Sign in with Google" button. We then widen the lens from websites to phone apps, installed programs, and embedded devices, and finish with where AI services fit in the picture.

BY THE END OF THIS SESSION STUDENTS CAN

- Distinguish authentication from authorization with examples.
- Explain why passwords are hashed and what multi-factor authentication adds.
- Describe how a cookie keeps a user logged in across stateless requests.
- Explain HTTP Basic Authentication and why it is only acceptable over HTTPS.
- Walk through the OAuth authorization-code flow and explain what it protects.
- Compare web, native mobile, cross-platform, desktop, and embedded software and describe how all of them use the same back end.
- Explain where an AI model such as Claude sits in a product's architecture.

SECTION	MINUTES
<u>5.1 Authentication versus authorization</u>	8
<u>5.2 Passwords done right</u>	8
<u>5.3 Staying logged in: cookies, sessions, and tokens</u>	10
<u>5.4 HTTP Basic Authentication and API keys</u>	7
<u>5.5 OAuth: how "Sign in with Google" works</u>	15
<u>5.6 Security basics every product owner should know</u>	7
Review questions and buffer	5

SECTION	MINUTES
<u>5.7 Beyond the browser: mobile, desktop, embedded</u>	10
<u>5.8 Where AI fits</u>	5
Review questions and buffer	5

5.1 Authentication versus authorization 8 MIN

Two words that sound alike and are constantly confused, even by professionals. Keep them apart with two questions.

- **Authentication** answers "*Who are you?*" It is the act of proving identity: a password, a code from your phone, a fingerprint, or a login through Google. Shortened to "authn" or just "auth".
- **Authorization** answers "*What are you allowed to do?*" Once the system knows you are Dana, may Dana see order 90? May Dana see *Sam's* order 88? May Dana issue a refund? Shortened to "authz".

ANALOGY

An office building. Showing your badge at the front desk is authentication: they confirm you are an employee. Which doors your badge opens is authorization: the server room is not among them unless you are in IT. A guest badge authenticates you as a visitor and authorizes almost nothing.

Authorization is usually organised through **roles**: customer, staff, manager, administrator. Rules are written per role ("staff may mark orders ready; only managers may refund") and users are assigned roles. Deciding the roles and rules of a product is a business question, not a technical one, and it is one you should expect to answer when you build something.

A rule of thumb that catches most authorization bugs in the wild: **every request that touches a specific record must check that *this* user may touch *this* record**. Systems that check "is the user logged in?" but forget "is this their order?" allow anyone to read anyone's data by changing the number in the URL. This exact flaw has produced many of the largest data leaks of the last decade.

5.2 Passwords done right 8 MIN

The oldest authentication method is a shared secret: the user and the server both know the password. The obvious design, storing each user's password in the database next to their email, is also the most dangerous thing a product can do. If the database is ever stolen (and databases are stolen constantly), every password is exposed, and since people reuse passwords, so are their bank and email accounts.

The correct design stores a [hash](#) of the password instead. A hash function turns any input into a fixed-length scramble in a way that cannot be reversed. `correct-horse-battery` might hash to `$2b$12$K9xT...`. To check a login, the server hashes what the user typed and compares the two scrambles. The server never needs the original password, and a thief who steals the database gets scrambles they cannot turn back into passwords. (Details you may hear: passwords are "salted," meaning a random value is mixed in so identical passwords do not produce identical hashes, and slow algorithms such as bcrypt or argon2 are used so that guessing billions of passwords takes years.)

The lesson for a product owner is simple and non-negotiable: **if a system can email you your old password, it is storing passwords wrongly**. A correctly built system can only reset a password, never reveal it.

Passwords alone are weak because people choose bad ones and reuse them. [Multi-factor authentication](#) (MFA, or two-factor, 2FA) adds a second, independent proof: something you *have* (a code from an authenticator app or a hardware key) or something you *are* (a fingerprint or face) on top of something you *know* (the password). A stolen password alone then no longer opens the account. Every serious product should offer it, and every business should require it for staff accounts. Newer *passkeys* go further and replace passwords with a cryptographic key stored on your device, unlocked by fingerprint or face; they are becoming the default on major platforms.

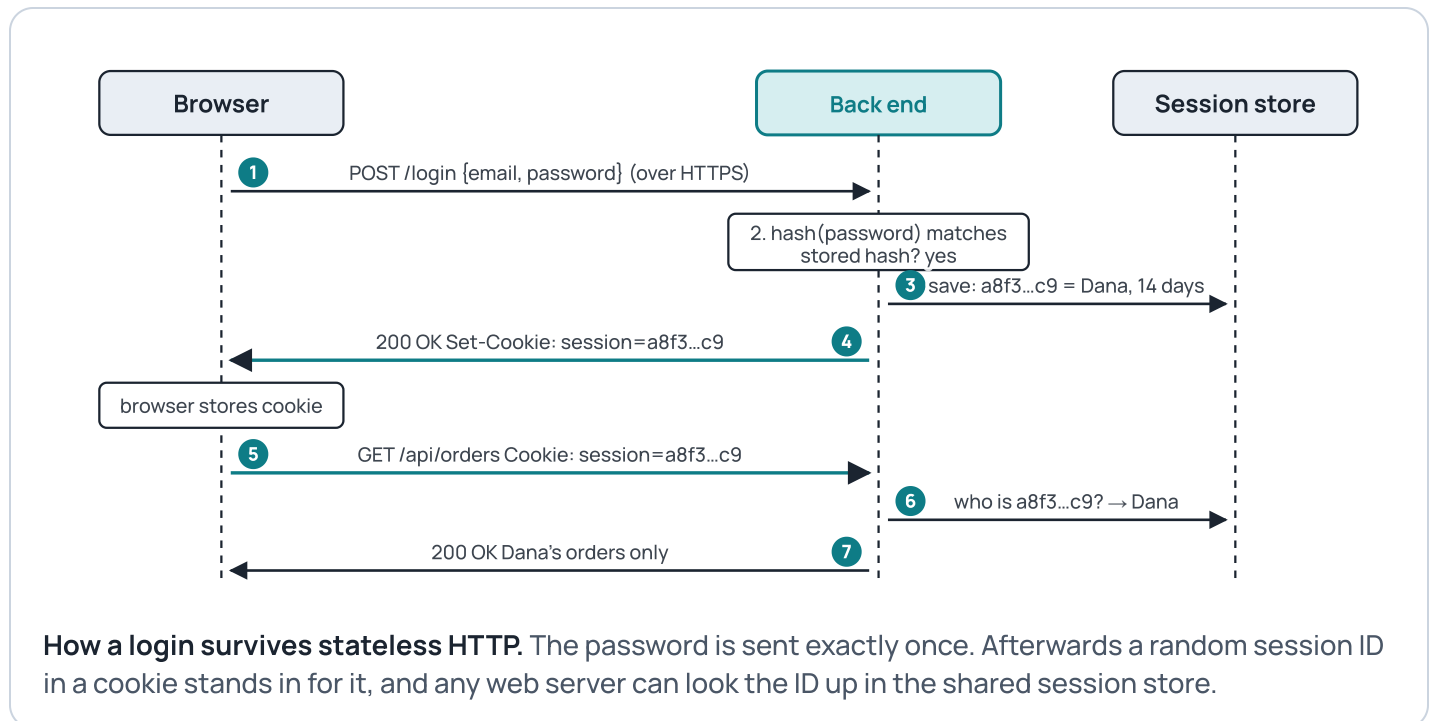
5.3 Staying logged in: cookies, sessions, and tokens

10 MIN

Session 4 established that HTTP is stateless: each request arrives with no memory of the last. Yet you log in once and stay logged in for weeks. How does the server recognise you on the second request?

The mechanism is the [cookie](#). When you log in successfully, the server's response includes an instruction: "browser, please store this piece of text and send it back to me with every future request." The browser obeys automatically. The text is usually a [session ID](#): a long random string that means nothing by itself but which the server has recorded in its session store

(the Redis box in Session 4) alongside "this is Dana, logged in at 8:14, expires in 14 days." Each request carries the cookie; the server looks up the ID; now it knows who is asking. Log out, and the server deletes the record so the ID becomes worthless.



Three consequences worth drawing out:

- **The cookie is as good as the password while it lasts.** Anyone who copies the session cookie can act as Dana until it expires. This is why cookies are marked to be sent only over HTTPS and never be readable by JavaScript, and why sensitive actions (changing the password, paying) often ask for the password again.
- **This is how any web server in a load-balanced pool can serve Dana.** The identity travels with the request; the facts live in the shared store. Stateless servers, from Session 4, depend on exactly this.
- **Cookies are also how advertising tracks people.** The same mechanism that remembers your login lets a third-party advertising server recognise your browser across every site that shows its ads. Cookie-consent banners and privacy laws exist because of that second use, not the first.

Mobile apps and APIs often use a variant: instead of a random ID that the server looks up, the server issues a **token**: a signed piece of text that itself says "Dana, valid until Friday" and cannot be forged because it carries a signature only the server could produce. The most common format is the JSON Web Token (JWT, said "jot"). The client sends the token in an

`Authorization` header rather than a cookie. The concept is identical: prove who you are once, receive a credential, present the credential thereafter.

5.4 HTTP Basic Authentication and API keys 7 MIN

Before cookies and tokens there was the simplest possible scheme, built into HTTP itself and still in use: [HTTP Basic Authentication](#). When a server requires it, the browser pops up its own plain grey username/password box (you have probably seen one on a router or a staging site). The browser then attaches the credentials to *every* request:

```
GET /admin/reports HTTP/1.1
Host: www.rosenthalsbakery.com
Authorization: Basic ZGFuYTpjb3JyZWN0LWhvcnNl
```

That jumble after `Basic` is not encryption. It is [Base64](#) encoding of the text `dana:correct-horse`, and anyone can decode it in a second. Basic Auth therefore has two rules and two uses:

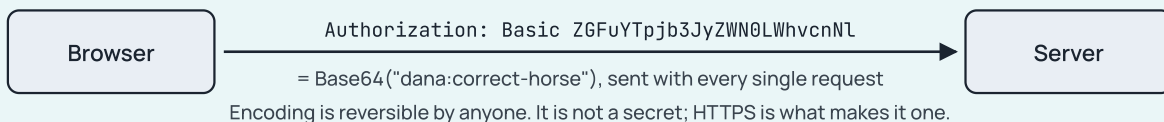
Rules

- Only ever over HTTPS. Over plain HTTP the password crosses the network in the clear on every request.
- Only for things where a shared, static password is acceptable. There is no logout, no expiry, no second factor.

Where you still meet it

- Putting a quick password in front of a staging site so the public cannot see it.
- Internal tools and admin pages for a handful of staff.
- Some APIs, where the "username" is an account ID and the "password" is a key. Stripe's API, for instance, accepts your secret key this way.

HTTPS tunnel: encrypted between browser and server. Without it, the header below is readable by anyone on the network.



Basic Authentication. The credentials ride along on every request in a trivially reversible encoding. The protection comes entirely from HTTPS.

A close cousin is the [API key](#), which we met in Session 3. It is a long random secret issued by a service provider that your back end presents with each call so the provider knows whose account to bill and what it may do. Keys are passwords for programs, and they must be treated as such: stored in server configuration (a file called `.env` is the convention), never typed into front-end code, and never committed into a Git repository. Automated bots scan GitHub around the clock for accidentally published keys and can spend thousands of dollars of someone's cloud budget within minutes. When you use Claude Code with the Claude API or any other service, this is the single most important habit to build.

5.5 OAuth: how "Sign in with Google" works 15 MIN

Two problems appear the moment products start working together. First, a new app wants to let users sign up without inventing yet another password, so it offers "Sign in with Google" or "Continue with Apple." Second, an app wants to *use* another service on the user's behalf: a scheduling tool that reads your Google Calendar, an accounting app that pulls transactions from your bank, a Claude integration that reads your Notion pages. The naive solution to both, asking the user to type their Google password into the new app, is catastrophic: the app would then hold a password that opens everything, forever, and the user could revoke it only by changing the password everywhere.

[OAuth 2.0](#) solves both problems with one idea: the user never gives the app their password. Instead, the user is sent to the service they already trust (Google), approves a specific, limited request there, and the trusted service hands the app a special-purpose key.

ANALOGY

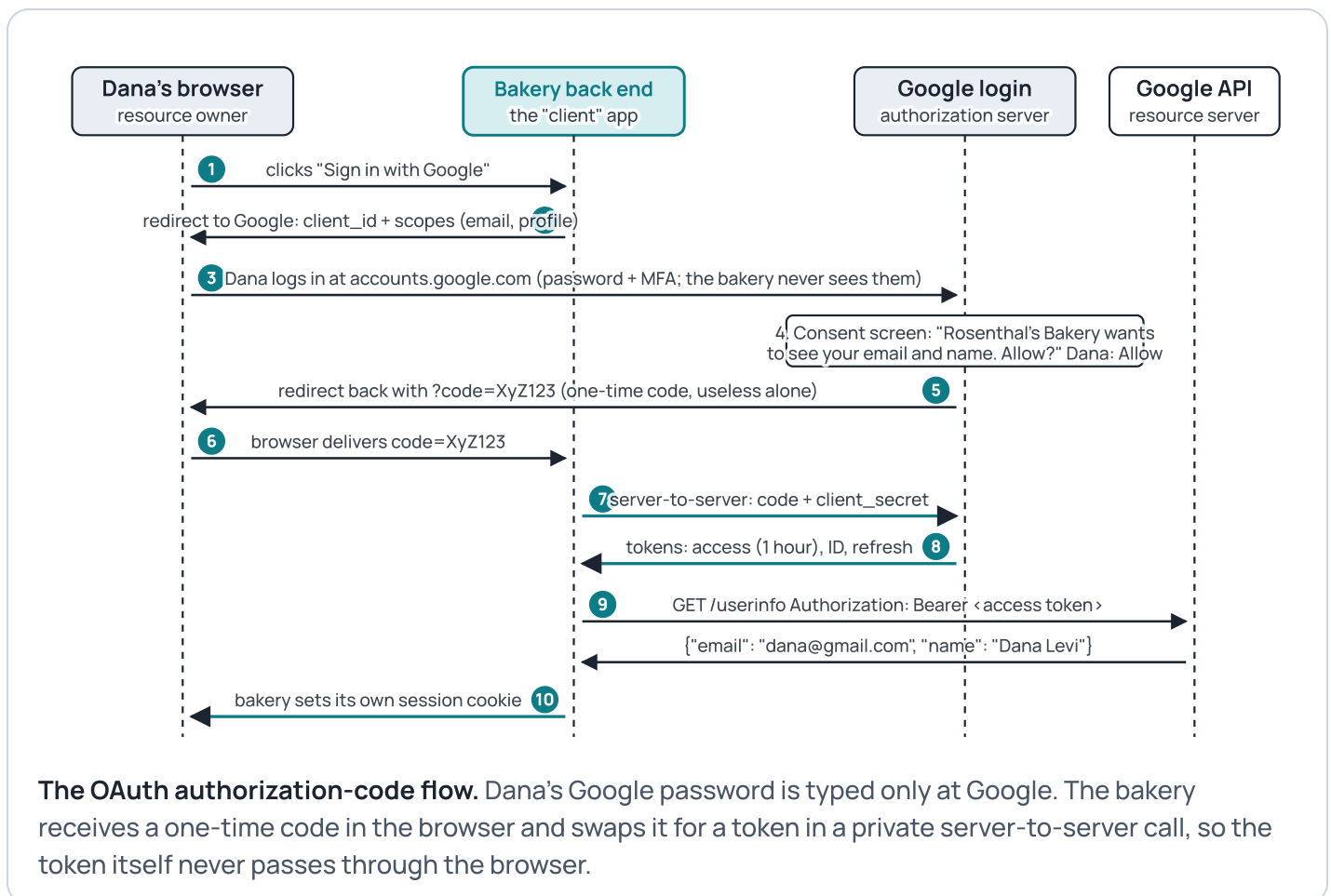
A valet key. You would not hand a valet the master key to your car, house, and safe. A valet key starts the engine, opens the driver's door, and does nothing else; you can have it cancelled without changing any other lock. OAuth issues valet keys for online accounts. The key is called an *access token*; the "engine only" restriction is called a *scope*.

The four parties

OAuth conversations always involve four roles. Learning their names makes every OAuth diagram readable.

ROLE	WHO IT IS	IN "SIGN IN WITH GOOGLE" FOR THE BAKERY APP
Resource owner	The user, who owns the data or identity	Dana
Client	The application that wants access (a different sense of "client" than in Session 2)	The bakery's web app
Authorization server	The trusted service that authenticates the user and issues tokens	Google's login and consent system
Resource server	The service holding the data the token unlocks	Google's profile or Calendar API

The authorization-code flow, step by step



Walk the class through it slowly; it is the most intricate diagram in the module and worth the time.

1. **Steps 1–3.** The bakery's app does not show a password box. It sends Dana's browser to Google, telling Google which app is asking (a public *client ID* the

bakery registered with Google beforehand) and what it wants (the [scopes](#) : here just email and name).

2. **Step 4.** Dana logs in *at Google*, with Google's password box and Google's MFA. The bakery sees none of it. Google shows a consent screen listing exactly the scopes requested. Dana can refuse.
3. **Steps 5–6.** Google sends the browser back to a pre-registered address at the bakery with a one-time *authorization code*. The code is deliberately useless on its own: it expires in a minute and can be redeemed only by the party holding the matching secret.
4. **Steps 7–8.** The bakery's back end, in a direct server-to-server call the browser never sees, presents the code together with its *client secret* (a password that only the bakery's server and Google know). Google returns the [access token](#) , valid for about an hour, plus a longer-lived *refresh token* for obtaining new ones, plus an ID token describing who Dana is.
5. **Steps 9–10.** The bakery uses the access token to ask Google for Dana's email and name, matches or creates her account, and issues its own ordinary session cookie. From here on, Session 5.3 applies.

Why the roundabout design? Because of what it protects against. Dana's password is never disclosed to the bakery. The token can do only what the scopes allow, so even a badly behaved app cannot read her email if it asked only for her name. Dana can see every app she has authorised in her Google account settings and revoke any of them with one click, instantly, without changing her password. And the access token expires quickly, so a stolen one has a short life. Compare each of those properties with the "type your Google password into our app" alternative and the value of the standard becomes obvious.

Login versus access: OpenID Connect

Strictly, OAuth is about *authorization*: granting an app access to resources. Using it for *login* (proving who the user is) was such a common desire that a thin standard was added on top: [OpenID Connect](#) (OIDC). It defines the ID token in step 8 and what it must contain. Every "Sign in with Google / Apple / Microsoft / GitHub" button is OIDC over OAuth. When a vendor says they support [single sign-on](#) (SSO), they mean a company's employees can log into the vendor's product with the company's central identity provider (Google Workspace, Microsoft Entra, Okta) using OIDC or its older enterprise cousin, SAML. For a business selling software to other businesses, supporting SSO is often a condition of the sale.

BUSINESS TRANSLATION

- Offering "Sign in with Google" raises sign-up conversion and removes your password-storage liability. Most new products should offer it alongside email and password.
- When another app asks to "connect" to your Google, Slack, or bank account, read the scopes on the consent screen. That list is the entire extent of what it can do.
- Your own product can be the resource server: letting partners "connect to your app" via OAuth is how platforms such as Shopify and Slack built their ecosystems.

5.6 Security basics every product owner should know

7 MIN

You do not need to be a security engineer, but a handful of ideas will let you ask the right questions and recognise the most common mistakes, several of which have already appeared in this module.

- **Encrypt in transit and at rest.** HTTPS everywhere (in transit); encrypted disks and database volumes (at rest). Every cloud provider offers the latter as a checkbox. Turn it on.
- **Never trust the front end.** Anything in the browser or the app can be read and altered by the user. Prices, discounts, permissions, and secrets belong on the server. This was the lesson of the three-tier diagram in Session 3.
- **Watch for injection.** The most famous web vulnerability, [SQL injection](#), happens when text a user types is pasted straight into a SQL query. If a login form builds `SELECT * FROM users WHERE name = ' + whatever was typed + '`, then typing `' OR 1=1 --` makes the query true for every user. The fix (parameterised queries, where user input is passed to the database separately from the SQL) is automatic in every modern framework and ORM, which is one reason to use them. A cousin, cross-site scripting (XSS), does the same trick with HTML and JavaScript. The general rule: user input is data, never code.
- **Least privilege.** Every person, program, and API key gets the minimum access it needs. The web server's database account should not be able to drop tables. An intern should not be an administrator. A leaked key that can only read is a nuisance; one that can do everything is a disaster.
- **Keep secrets out of code.** Passwords, API keys, and client secrets go in environment configuration, never in source files and never in Git. Rotate any secret that has ever been exposed.
- **Keep software updated and take backups.** Most breaches exploit known holes in unpatched software. Managed services (PaaS, managed databases)

patch for you, which is a security argument for them, not only a convenience one. And test that backups actually restore; an untested backup is a hope, not a plan.

The industry maintains a list of the ten most common web application risks, published by an organisation called OWASP. You do not need to memorise it, but "have we reviewed against the OWASP Top Ten?" is a question that any engineer or AI coding assistant can act on, and asking it marks you as someone who knows what matters.

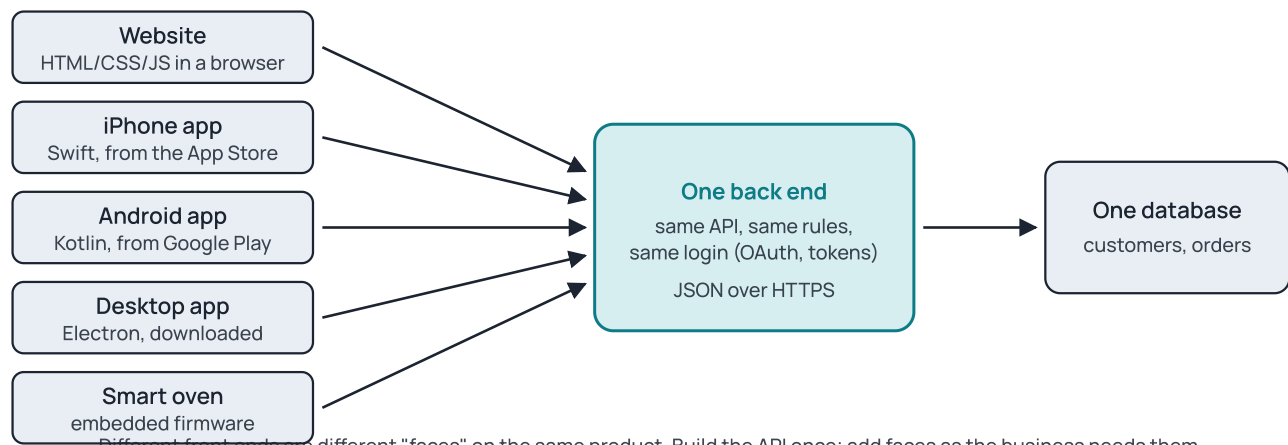
5.7 Beyond the browser: mobile, desktop, embedded

10 MIN

Everything so far has assumed the client is a browser. Most software you use is not a website, and yet almost all of it is built from the same parts. The distinction that matters is *where the front end runs*; the back end and the database are the same story.

KIND OF SOFTWARE	WHERE THE FRONT END RUNS	BUILT WITH	DISTRIBUTED HOW	TRADE-OFFS
Website / web app	Browser	HTML, CSS, JavaScript (often with a framework such as React)	Just a URL; nothing to install; updates instantly	Limited access to device hardware; needs a connection
<u>Native mobile app</u>	Phone or tablet, installed	Swift (iOS), Kotlin (Android)	Apple App Store, Google Play; store review; 15–30% commission on in-app purchases	Best performance, full hardware access, offline use; two separate codebases to maintain
<u>Cross-platform app</u>	Phone, installed	React Native, Flutter	App stores, same as native	One codebase for both platforms; slightly less polish and performance than native
<u>Progressive web app</u> (PWA)	Browser, but installable to home screen	Web technologies	URL; no store	No store commission or review; some features

KIND OF SOFTWARE	WHERE THE FRONT END RUNS	BUILT WITH	DISTRIBUTED HOW	TRADE-OFFS
				unavailable, especially on iPhone
<u>Desktop application</u>	Laptop or PC, installed	Native (C#, Swift, C++) or Electron (web technologies in a desktop wrapper: Slack, VS Code, Discord, Spotify)	Download and installer, Mac App Store, Microsoft Store; must ship its own updates	Full power of the machine, works offline; installation friction, per-OS testing
<u>Embedded software</u>	Inside a device: thermostat, car, payment terminal, medical device	C, C++, Rust; tiny processors, little memory	Shipped inside the hardware; updated over the air, if at all	Hard to change after sale; reliability and security are paramount; the Internet of Things (IoT) connects them to the cloud
Command-line tool	A terminal on a computer	Python, Go, Rust, JavaScript	Package managers (pip, npm, Homebrew)	Text only; favoured by developers; Claude Code itself is one



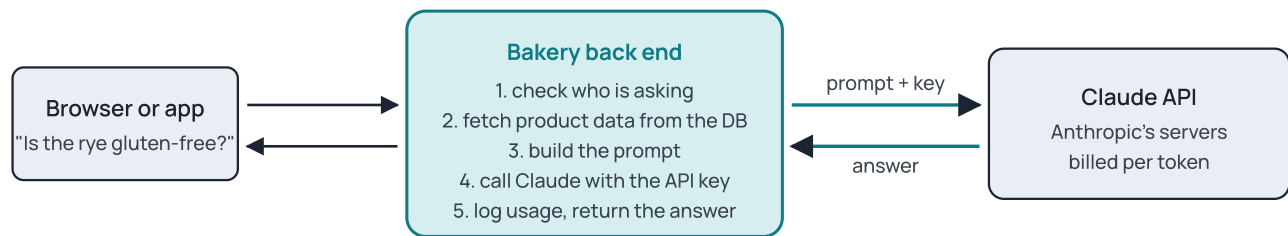
One back end, many clients. Whether the customer orders from a browser, a phone, a laptop, or the oven's own screen, the request reaches the same API and the same data. The architecture of Sessions 3 and 4 does not change.

This picture explains several things business people encounter:

- **Why "we need an app" is a smaller decision than it sounds.** If the back end and API already exist, a mobile app is a new front end, not a new system. If they do not exist, the app project is secretly also a back-end project.
- **Why companies start with the web.** A web app has no store review, no commission, instant updates, and one codebase. Native apps are added when the product needs the camera, notifications, offline use, or a home-screen presence.
- **Why the API is the product's most durable asset.** Front ends are rewritten every few years as fashions and devices change; a well-designed API and its database outlive several generations of them.
- **Why embedded devices are the exception.** A thermostat sold in 2026 must still work in 2036, may never be updated, and can physically harm someone if it fails. The engineering culture is correspondingly conservative, and the cost of a mistake is a recall rather than a redeploy.

5.8 Where AI fits 5 MIN

Since this course is about building products with AI, the last question is where a model such as Claude sits in everything above. The answer is reassuringly familiar: **an AI model is a third-party API called from your back end.** It is the Claude API box that has been sitting at the edge of the diagrams since Session 3.



The API key and the model call live on the server. The browser only ever talks to your own back end.

AI in the architecture. The model is one more service the back end calls, after applying its own rules and adding its own data. Nothing about the tiers, hosting, or security changes.

Three practical consequences:

- **The key stays on the server.** A model call from the browser would expose your API key to every visitor, who could run up your bill. All the reasoning of Section 5.4 applies.
- **Your data is the differentiator.** The model is the same one your competitors can rent. What makes an AI feature yours is the context your back end adds from your database (this customer's history, your product catalogue, your policies) before calling the model. That pattern, fetching relevant data and putting it in the prompt, is what people mean by **retrieval-augmented generation** (RAG).
- **AI calls have cost, latency, and failure modes.** A model call takes seconds and costs money per word, so it is treated like any slow, paid third-party API: cached where possible, monitored, and given a fallback. When Claude Code builds an AI feature for you, these are the questions to ask it.

Step back and look at the whole module. A model such as Claude, the tool you are learning to build with, is itself served through exactly the architecture described here: a front end (the Claude app, or Claude Code in your terminal), load-balanced back ends behind an API, databases, caches, OAuth login, and an API key for programs. The next time you use it, you will know what is on the other end of the request.



Session 5: Identity, security, and software beyond the browser

1. Define authentication and authorization, and give an example where a system gets the first right and the second wrong.
2. Why should a database store a hash of each password rather than the password itself? What does it mean if a company can email you your old password?
3. HTTP is stateless, yet you stay logged in. Explain the role of the cookie and the session store.
4. What does HTTP Basic Authentication send, in what form, and why is it acceptable only over HTTPS?
5. Name the four roles in OAuth and identify each one when a scheduling app asks to read your Google Calendar.
6. In the OAuth flow, why does Google send a one-time code back through the browser instead of the access token itself? And why does the user never type their Google password into the bakery's app?
7. What is a scope in OAuth, and where does the user see it?
8. What is SQL injection, and why does using a modern framework or ORM largely prevent it?
9. Your company has a web app with a working API. The CEO wants "an iPhone app and an Android app." In architectural terms, how big is that project, and what alternative might you propose first?
10. Where in a product's architecture does a call to an AI model such as Claude belong, and why must it not be made from the browser?

Answers are in [Appendix B, Session 5](#).

APPENDIX A

Keywords Students Should Be Able to Define

Every highlighted keyword in the module, alphabetically, with a one-line definition and a link to the passage where it is introduced. Suggested use: hand out as a study sheet, or quiz students on a random sample at the start of each session.

133 keywords.

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
A		
<u>access token</u>	The short-lived credential an OAuth authorization server issues to an application, which the application attaches to API requests to act within the granted scopes. Typically valid for about an hour; renewed using a longer-lived refresh token.	<u>5.6 Security basics every product owner</u>
<u>AI-assisted development</u>	Building software with an AI model doing much of the writing, explaining, and debugging of code under a person's direction, using tools such as Claude Code. Shifts the human's work from typing code toward specifying,	<u>1.7 Making the decision in the age of AI</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
	reviewing, testing, and deciding.	
<u>API</u>	Application Programming Interface. The defined set of requests a program accepts and the responses it gives, so that other programs can use it without knowing how it works inside. A web API is reached over HTTP and usually speaks JSON.	<u>3.2 APIs and JSON: how programs talk to</u>
<u>API key</u>	A long random secret string issued by a service to identify and bill the account calling its API. Sent with each request, kept on the server side of your product, never in front-end code or public repositories. Leaked keys are the most common cause of surprise cloud bills.	<u>5.5 OAuth: how "Sign in with Google" wor</u>
<u>architecture</u>	The overall structure of a piece of software: what its major parts are, what each part is responsible for, and how the parts communicate.	<u>Choosing a Programming Language</u>
<u>Authentication</u>	Verifying that someone is who they claim to be, typically by checking a	<u>5.2 Passwords done right</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
	password, a code from their phone, a fingerprint, or a token issued by a trusted identity provider. Answers 'Who are you?' Often shortened to 'authn'.	
<u>Authorization</u>	Deciding what an already-identified user is allowed to do: view this order, edit that page, approve a refund. Answers 'What may you do?' Often shortened to 'authz'.	<u>5.2 Passwords done right</u>
B		
<u>back end</u>	The part of an application that runs on servers, out of the user's sight: the code that applies business rules, talks to the database, processes payments, and answers the front end's requests.	<u>2.6 Front end, back end, and the limits of</u>
<u>Base64</u>	A way of writing any data using only letters, digits, and a few symbols so it can travel safely in text. It is encoding, not encryption: anyone can decode it instantly. Basic Auth uses it to pack	<u>5.4 HTTP Basic Authentication and API ke</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
	'username:password' into one header.	
<u>business logic</u>	The rules of the business as encoded in the back end: how prices are computed, who may see what, what makes an order valid, when to send a notification. Kept on the server so it cannot be bypassed or tampered with.	<u>3.6 Putting it together: the three-tier arc</u>
<u>bytecode</u>	An intermediate, platform-neutral form of a compiled program (Java's .class files, .NET's assemblies) that is run by a virtual machine installed on each platform, rather than directly by the processor.	<u>1.6 Platforms and portability</u>
C		

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
<u>cache</u>	A fast temporary copy of data that is expensive to compute or fetch, kept close to where it is needed so the next request for it is answered instantly. Caches exist in the browser, on servers (Redis), and in CDNs. The hard part is knowing when the copy is out of date.	<u>4.5 Step 4: caches, CDNs, and storage</u>
<u>CDN</u>	Content Delivery Network: a worldwide network of servers that keep copies of a site's static files (images, CSS, JavaScript, video) close to users, so pages load fast everywhere and the origin servers handle far less traffic. Cloudflare, Akamai, and Amazon CloudFront are examples.	<u>4.5 Step 4: caches, CDNs, and storage</u>
<u>CI/CD</u>	Continuous Integration / Continuous Deployment: automation that, every time code is changed, runs the tests and, if they pass, deploys the new version, so releases are frequent, small, and low-risk.	<u>4.7 Shipping code safely</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
	GitHub Actions is a common tool.	
<u>client</u>	The computer or program that asks for something, such as your laptop's browser requesting a web page.	<u>2.2 Clients, servers, and programs</u>
<u>client-server model</u>	The dominant pattern for networked software: many clients send requests to a server, and the server sends back responses.	<u>2.2 Clients, servers, and programs</u>
<u>cloud computing</u>	Renting computing (servers, databases, storage, networking) from a provider's data centers over the internet, paid by usage, instead of buying and running your own hardware. The big three providers are Amazon Web Services, Microsoft Azure, and Google Cloud.	<u>4.6 The cloud menu: IaaS, PaaS, SaaS, se</u>
<u>compiler</u>	A program that translates source code into machine code ahead of time, producing an executable file for a specific kind of processor and operating system. C, C++, Rust, Go, and Swift are compiled	<u>1.1 What a programming language is, and</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
	languages. The result runs fast but must be rebuilt for each platform.	
<u>container</u>	A standard, self-contained package holding a program together with everything it needs to run (language runtime, libraries, settings), so that it runs identically on a laptop, a test server, and production. Docker is the tool that creates them; Kubernetes runs many of them across many machines.	<u>4.7 Shipping code safely</u>
<u>cookie</u>	A small piece of text a server asks the browser to store and to send back automatically with every future request to that site. The standard way a website recognises a returning browser, most importantly to keep you logged in.	<u>5.3 Staying logged in: cookies, sessions,</u>
<u>Cross-platform app</u>	A way of building mobile (and sometimes desktop) apps from one codebase that runs on both iPhone and Android, using frameworks such as	<u>5.7 Beyond the browser: mobile, desktop</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
	React Native or Flutter. Saves building the same app twice at some cost in performance or platform polish.	
<u>Cross-platform frameworks</u>	A toolkit (React Native, Flutter, .NET MAUI) that lets one codebase produce apps for both iOS and Android, and sometimes desktop, by translating to each platform's native components or drawing its own. Saves writing the app twice at some cost in performance and platform polish.	<u>1.7 Making the decision in the age of AI</u>
<u>CRUD</u>	Create, Read, Update, Delete: the four basic operations any application performs on stored data. Most business software is, at heart, CRUD screens over a database.	<u>3.2 APIs and JSON: how programs talk to</u>
<u>CSS</u>	Cascading Style Sheets. The language that controls the visual appearance of HTML: colors, fonts, spacing, layout, and how the page adapts to different screen sizes.	<u>2.3 A web page is three languages</u>
D		

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
<u>database</u>	A program dedicated to storing, organising, and retrieving data reliably, so that many users and programs can read and write it at the same time without losing or corrupting anything. Almost every dynamic application has one.	<u>3.3 Why every product needs a database</u>
<u>database management system</u>	Database Management System: the actual software product that implements a database, such as PostgreSQL, MySQL, SQLite, SQL Server, Oracle, or MongoDB.	<u>3.3 Why every product needs a database</u>
<u>database server</u>	A machine (or managed cloud service) whose only job is to run the database, separate from the servers that run the application. Allows the two to be sized, secured, and scaled independently.	<u>4.3 Step 2: give the database its own machine</u>
<u>data center</u>	A building full of servers with reliable power, cooling, and fast internet connections. Cloud providers run enormous data centers and rent out slices of them.	<u>2.2 Clients, servers, and programs</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
<u>deployment</u>	The act of putting a version of your software onto servers where real users can reach it. 'We deployed on Tuesday' means Tuesday's version went live.	<u>4.1 From localhost to production</u>
<u>Desktop application</u>	A program installed on a laptop or desktop computer (Windows, macOS, Linux). Built natively or with web technologies wrapped in a desktop shell such as Electron (Slack, VS Code, Discord).	<u>5.7 Beyond the browser: mobile, desktop</u>
<u>DNS</u>	Domain Name System. The internet's directory service that translates a domain name (rosenthalsbakery.com) into the IP address of the server that hosts it. The first step in loading any website.	<u>2.5 How the internet delivers a page</u>
<u>domain name</u>	The human-readable name of a website, such as rosenthalsbakery.com. Domains are rented from a registrar, usually by the year, and are translated into IP addresses by DNS.	<u>2.5 How the internet delivers a page</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
<u>domain-specific languages</u>	A language built for one narrow job and unsuitable for others: SQL for querying databases, HTML for describing page structure, CSS for styling, regular expressions for matching text patterns.	<u>1.2 Fit for purpose: what each language v</u>
<u>dynamic website</u>	A website whose pages are produced by a program on the server at the moment of each request, so that content can depend on who is asking, what they asked for, and what is stored in a database.	<u>2.6 Front end, back end, and the limits of</u>
E		
<u>ecosystem</u>	Everything that surrounds a language beyond the language itself: its libraries and frameworks, the package manager that distributes them, its tools, hosting support, documentation, and community. Often a bigger factor in language choice than the language's own features.	<u>1.4 The ecosystem: packages and libraries</u>
<u>Embedded software</u>	Software built into a physical device to	<u>5.7 Beyond the browser: mobile, desktop</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
	control it: a thermostat, a car's dashboard, a payment terminal, a smart speaker. Runs on small, constrained processors, is hard to update, and increasingly reports to and takes commands from cloud servers (the Internet of Things).	
<u>encryption</u>	Scrambling data with a mathematical key so that only someone holding the matching key can read it. Used in transit (HTTPS) and at rest (encrypted disks and databases).	<u>2.6 Front end, back end, and the limits of</u>
<u>endpoints</u>	One specific address in an API, such as GET /api/products or POST /api/orders, that performs one operation. An API is a collection of endpoints.	<u>3.2 APIs and JSON: how programs talk to</u>
<u>environments</u>	A complete, separate copy of the application and its supporting services used for one purpose. Development (the programmer's own machine), staging (a private replica of production for testing), and production (the live system real users	<u>4.2 Step 1: everything on one server</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
	use) are the standard three.	
F		
<u>firewall</u>	A rule set on a network or server that decides which connections are allowed in or out, for example 'accept web traffic on port 443 from anywhere; accept database connections on port 5432 only from the web server'.	<u>4.4 Step 3: many web servers behind a load balancer</u>
<u>foreign key</u>	A column in one table that holds the primary key of a row in another table, linking the two. The customer_id in an orders table is a foreign key pointing to the customers table.	<u>3.3 Why every product needs a database</u>
<u>framework</u>	A pre-built collection of code that handles the routine parts of a kind of program (for a web framework: receiving requests, matching paths to functions, sending responses) so that developers write only the parts specific to their product. Flask, Django, FastAPI, Express, Rails, and Next.js are web frameworks.	<u>3.1 Inside the back end: a Python web server</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
<u>front end</u>	The part of an application that runs on the user's device and that the user sees and interacts with: the HTML, CSS, and JavaScript in the browser, or the screens of a mobile app.	<u>2.6 Front end, back end, and the limits of</u>
<u>full-stack</u>	Describes a developer or a project that covers both the front end (what the user sees) and the back end (servers and databases). 'The stack' is the set of technologies a product is built from.	<u>2.7 Where Python fits</u>
G		
<u>general-purpose languages</u>	A programming language designed to write any kind of program: Python, JavaScript, Java, C#, C++, Go, Rust. Contrast with a domain-specific language, built for one job.	<u>1.2 Fit for purpose: what each language v</u>
H		
<u>hallucinate</u>	An AI model producing confident but false output, such as a function, package, or API that does not exist. In coding, the usual sign is an import or	<u>1.5 Speed and memory: when performanc</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
	install command that fails. Always verify that a suggested package is real and maintained.	
<u>hash</u>	A fixed-length scramble of some input produced by a one-way mathematical function: the same input always gives the same hash, but the hash cannot be reversed to recover the input. Passwords are stored as hashes so that a stolen database does not reveal them.	<u>5.3 Staying logged in: cookies, sessions,</u>
<u>health checks</u>	A small request a load balancer sends to each server every few seconds to confirm it is alive and responding correctly. A server that fails its health check is removed from rotation until it recovers.	<u>4.4 Step 3: many web servers behind a lo</u>
<u>high availability</u>	A design in which every component has at least one duplicate ready to take over, so that the failure of any single machine does not interrupt the service. The way three- and four-nines uptime is achieved.	<u>4.5 Step 4: caches, CDNs, and storage</u>
<u>Horizontal scaling</u>	Adding capacity by running more copies of	<u>4.4 Step 3: many web servers behind a lo</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
	a server side by side and spreading requests across them. No ceiling in principle, and losing one copy does not take the service down. Requires a load balancer and stateless servers.	
<u>Hosting</u>	Paying a company to keep your website's files and programs on servers that are always on and connected to the internet, so that anyone can reach them.	<u>2.6 Front end, back end, and the limits of</u>
<u>HTML</u>	HyperText Markup Language. The language that describes the content and structure of a web page: headings, paragraphs, images, links, forms. Every web page is HTML at its core.	<u>2.3 A web page is three languages</u>
<u>HTTP</u>	HyperText Transfer Protocol. The set of rules browsers and servers use to talk: the client sends a request naming a method (such as GET) and a path, and the server sends back a response with a status code and, usually, content.	<u>2.5 How the internet delivers a page</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
<u>HTTP Basic Authentication</u>	The simplest authentication method built into HTTP: the client sends the username and password, encoded (not encrypted) in Base64, in an Authorization header on every request. Acceptable only over HTTPS, and used mainly for internal tools, quick protection of staging sites, and some APIs.	<u>5.4 HTTP Basic Authentication and API ke</u>
<u>HTTPS</u>	HTTP Secure: HTTP carried over an encrypted connection (TLS), so that nobody between the browser and the server can read or alter the traffic. Shown as a padlock in the browser. Standard for all websites today.	<u>2.6 Front end, back end, and the limits of</u>
I		
<u>IaaS</u>	Infrastructure as a Service: renting raw computing (virtual machines, disks, networks). You install and run everything on them yourself. Maximum control, maximum work. Amazon EC2, DigitalOcean.	<u>4.6 The cloud menu: IaaS, PaaS, SaaS, se</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
<u>interpreter</u>	A program that reads source code and carries out its instructions on the spot, without producing an executable in advance. Python, JavaScript, and Ruby are interpreted. Slower than compiled code but portable: the same source runs anywhere the interpreter is installed.	<u>1.2 Fit for purpose: what each language v</u>
<u>IP address</u>	Internet Protocol address, the numeric address of a computer on a network, such as 142.250.72.14. Every server reachable on the internet has one. DNS translates domain names into IP addresses.	<u>2.5 How the internet delivers a page</u>
J		
<u>JavaScript</u>	The programming language that runs inside the browser and makes web pages interactive: reacting to clicks, validating forms, fetching new data without reloading the page. Not related to Java despite the name.	<u>2.3 A web page is three languages</u>
<u>JOIN</u>	The SQL operation that combines rows from	<u>3.4 SQL by example</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
	two tables by matching a foreign key in one to the primary key in the other, for example attaching each order to the customer who placed it.	
<u>JSON</u>	JavaScript Object Notation. The standard text format for exchanging data between programs over the web: values in curly braces with 'name': value pairs and square brackets for lists. Human-readable and understood by every programming language.	<u>3.2 APIs and JSON: how programs talk to</u>
L		
<u>Latency</u>	The delay between a request being sent and the response arriving, usually measured in milliseconds. Users notice anything over about 100 ms; a page taking more than 3 seconds loses a large fraction of visitors.	<u>4.2 Step 1: everything on one server</u>
<u>library</u>	A collection of ready-made code that a program can call to do a specific job (send email, draw charts, talk to a database) so that	<u>1.2 Fit for purpose: what each language w</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
	the developer does not write it from scratch. Also called a package or module when distributed through a package manager.	
<u>load balancer</u>	A component that receives all incoming requests and distributes them across a pool of identical servers, so that no one server is overloaded and a failed server is skipped automatically. The public address of a scaled website points at its load balancer.	<u>4.4 Step 3: many web servers behind a load balancer</u>
<u>localhost</u>	The name a computer uses for itself. Opening <code>http://localhost:5000</code> in a browser connects to a server program running on your own machine, which is how developers test before deploying.	<u>3.2 APIs and JSON: how programs talk to each other</u>
M		

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
<u>machine code</u>	The numeric instructions a processor executes directly. Nobody writes it by hand any more; compilers and interpreters produce it from human-readable source code.	<u>1.1 What a programming language is, and</u>
<u>managed database</u>	A database run for you by a cloud provider (Amazon RDS, Google Cloud SQL, Supabase, Neon, PlanetScale): they handle installation, backups, security patches, and failover; you get a connection address and a monthly bill.	<u>4.4 Step 3: many web servers behind a load balancer</u>
<u>maturity</u>	How long a language or tool has been in serious use, how stable and well-documented it is, how many known problems have been fixed, and how confident one can be that it will still be supported in ten years.	<u>1.4 The ecosystem: packages and libraries</u>
<u>Memory footprint</u>	How much of a computer's working memory (RAM) a program uses while running. Matters on small devices, on phones, and on servers	<u>1.6 Platforms and portability</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
	where memory is what you pay for; matters little for most business applications on modern hardware.	
<u>method</u>	The verb at the start of an HTTP request that says what kind of action is wanted. GET reads, POST creates or submits, PUT/PATCH updates, DELETE removes.	<u>3.2 APIs and JSON: how programs talk to</u>
<u>Monitoring</u>	Continuously collecting measurements and log messages from a running system (errors, response times, traffic, server health) and alerting people when something goes wrong, so problems are found before customers report them.	<u>4.7 Shipping code safely</u>
<u>Multi-factor authentication</u>	Multi-Factor Authentication: requiring two or more independent proofs of identity, typically something you know (a password) plus something you have (a code from your phone, a hardware key) or something you are (a fingerprint). Blocks most account takeovers	<u>5.3 Staying logged in: cookies, sessions,</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
	even when the password is stolen.	
N		
<u>Native mobile app</u>	A mobile or desktop application written in the platform's own language and tools (Swift for iPhone, Kotlin for Android) and installed on the device. Fastest and most capable on that platform, but a separate codebase per platform.	<u>5.7 Beyond the browser: mobile, desktop</u>
<u>NoSQL</u>	A family of databases that do not use the table-and-SQL model. Document databases (MongoDB) store JSON-like records with flexible shapes; key-value stores (Redis) hold simple lookups very fast. Chosen for flexibility or speed at very large scale.	<u>3.6 Putting it together: the three-tier arc</u>
O		
<u>OAuth 2.0</u>	OAuth 2.0: the standard protocol that lets a user grant one application limited access to their account at another service (or prove their identity to it) without ever sharing their password. The user is	<u>5.5 OAuth: how "Sign in with Google" wor</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
	sent to the trusted service to approve specific permissions, and the application receives a scoped, revocable access token instead of the password. Behind every 'Sign in with Google' button.	
<u>object storage</u>	A cloud service for storing files of any size (images, documents, video, backups) at very low cost and reachable from anywhere by URL. Amazon S3 is the original; every cloud has an equivalent. The standard home for user uploads.	<u>4.5 Step 4: caches, CDNs, and storage</u>
<u>OpenID Connect</u>	OpenID Connect: a layer on top of OAuth 2.0 that standardises using it for login. The authorization server returns an ID token stating who the user is, which is what 'Sign in with Google/Apple/Microsoft' actually uses.	<u>5.6 Security basics every product owner</u>
<u>open source</u>	Software whose source code is published for anyone to read, use, modify, and redistribute, usually	<u>1.4 The ecosystem: packages and libraries</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
	free of charge under a licence. Most programming languages, libraries, and much infrastructure (Linux, PostgreSQL) are open source.	
<u>operating system</u>	The base software that manages a computer's hardware and runs programs: Linux on almost all servers, Windows and macOS on desktops, iOS and Android on phones.	<u>4.2 Step 1: everything on one server</u>
<u>ORM</u>	Object-Relational Mapper: a library that lets a program work with database rows as ordinary objects in its own language (Python, JavaScript) and writes the SQL behind the scenes. SQLAlchemy, Django ORM, and Prisma are examples.	<u>3.6 Putting it together: the three-tier arc</u>
P		
<u>PaaS</u>	Platform as a Service: you hand the provider your code and they run it, handling servers, operating systems, scaling, HTTPS, and often the database. Less control, far less work.	<u>4.6 The cloud menu: IaaS, PaaS, SaaS, se</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
	Heroku, Render, Railway, Vercel, Fly.io.	
<u>package manager</u>	A tool that downloads, installs, and updates libraries for a language from a central registry, and records which ones a project depends on. pip and uv for Python, npm for JavaScript, Maven for Java, cargo for Rust.	<u>1.4 The ecosystem: packages and libraries</u>
<u>path</u>	The part of a URL after the domain, such as /menu.html or /products/42, that identifies which page, file, or resource on the server is being requested.	<u>2.5 How the internet delivers a page</u>
<u>platform</u>	The combination of hardware and operating system a program runs on: Windows on an Intel laptop, macOS on Apple silicon, Linux on a server, iOS on an iPhone, Android on a Samsung phone, or a browser. Software must be built or packaged separately for each platform it targets unless something in between hides the difference.	<u>1.6 Platforms and portability</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
<u>port</u>	A numbered 'door' on a computer that a server program listens on, so that one machine can run several services. Web traffic uses port 80 (HTTP) and 443 (HTTPS); developers often run test servers on ports like 3000, 5000, or 8000.	<u>3.2 APIs and JSON: how programs talk to</u>
<u>primary key</u>	A column whose value uniquely identifies each row in a table, such as customer_id. No two rows share one, and it never changes, so it can be used to refer to that row from anywhere.	<u>3.3 Why every product needs a database</u>
<u>private network</u>	A network inside a data center or cloud account that is not reachable from the public internet. Databases and internal services are placed on it so that only your own servers can connect to them.	<u>4.4 Step 3: many web servers behind a lo</u>
<u>processor architecture</u>	The family of a processor, which determines the machine code it understands. The two that matter today are x86 (Intel and AMD, most PCs and servers) and ARM (all phones,	<u>1.6 Platforms and portability</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
	Apple silicon Macs, and a growing share of servers). Compiled programs must be built separately for each.	
<u>programming language</u>	A precise, human-readable notation for giving instructions to a computer. Programs written in it (source code) are translated into machine code by a compiler or run by an interpreter. Python, JavaScript, Java, C++, and SQL are programming languages.	<u>1.1 What a programming language is, and</u>
<u>Progressive web app</u>	Progressive Web App: a website built so that it can be installed to the home screen, work offline, and send notifications, blurring the line between website and app. Distributed by URL, not through an app store.	<u>5.7 Beyond the browser: mobile, desktop</u>
<u>protocol</u>	An agreed set of rules for how two computers exchange messages, so that programs written by different people can still understand each other. HTTP, HTTPS, and DNS are all protocols.	<u>2.5 How the internet delivers a page</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
Q		
<u>query</u>	A question or instruction sent to a database, written in SQL. 'Running a query' means asking the database for data or telling it to change data.	<u>3.4 SQL by example</u>
<u>query string</u>	The optional part of a URL after a question mark, made of name=value pairs such as ?lang=en&page=2, used to pass extra details to the server.	<u>2.5 How the internet delivers a page</u>
R		
<u>read replica</u>	A live copy of the database that is continuously kept up to date from the primary. Read replicas answer read-only queries to take load off the primary; a standby replica takes over if the primary fails.	<u>4.5 Step 4: caches, CDNs, and storage</u>
<u>relational model</u>	The way of organising data into tables of rows and columns, where each table describes one kind of thing (customers, orders) and tables are linked to each other by shared identifying values	<u>3.3 Why every product needs a database</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
	(keys). Managed by relational databases and queried with SQL.	
<u>rendering</u>	The process by which a browser turns HTML, CSS, and JavaScript into the visible page on screen.	<u>2.5 How the internet delivers a page</u>
<u>request</u>	The message a client sends to a server asking for something. An HTTP request names a method (GET, POST, and so on), a path, and headers with extra details, and may carry a body of data.	<u>2.5 How the internet delivers a page</u>
<u>response</u>	The message a server sends back to a client after a request. An HTTP response carries a status code (200 OK, 404 Not Found), headers, and usually a body such as an HTML page or JSON data.	<u>2.5 How the internet delivers a page</u>
<u>REST</u>	A widely used style for designing web APIs in which each URL names a resource (a product, an order) and the HTTP method (GET, POST, PUT, DELETE) says what to do with it. 'A REST API' is the most common kind of API a	<u>3.2 APIs and JSON: how programs talk to</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
	business will build or consume.	
<u>retrieval-augmented generation</u>	Retrieval-Augmented Generation: having the back end look up relevant information (from a database, documents, or search) and include it in the prompt, so the AI model answers using your data rather than only what it learned in training.	<u>5.8 Where AI fits</u>
<u>roles</u>	A named bundle of permissions assigned to users, such as customer, staff, or administrator, so that access rules are written once per role instead of once per person. Role-based access control (RBAC) is the common name for this approach.	<u>5.2 Passwords done right</u>
<u>route</u>	A rule in a back-end program that says which function should run when a request arrives for a particular path, such as /hours or /orders/42. Routing is the act of matching incoming paths to code.	<u>3.2 APIs and JSON: how programs talk to</u>
S		

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
<u>SaaS</u>	Software as a Service: finished software you use through a browser or app and pay for by subscription, with nothing to install or run. Gmail, Salesforce, Shopify, Slack, Notion.	<u>4.7 Shipping code safely</u>
<u>schema</u>	The blueprint of a database: which tables exist, what columns each has, what type of data each column holds, and how tables link to one another through keys.	<u>3.4 SQL by example</u>
<u>scopes</u>	In OAuth, a named permission the application asks for, such as 'read email address' or 'read calendar events'. The user sees the scopes on the consent screen and the issued token can do nothing beyond them.	<u>5.5 OAuth: how "Sign in with Google" wor</u>
<u>server</u>	A computer, or a program on a computer, that waits for requests from clients and answers them. Servers are ordinary computers, usually in a data center, running around the clock.	<u>2.2 Clients, servers, and programs</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
<u>Serverless</u>	A cloud model in which you upload individual functions and the provider runs each one only when a request arrives, charging per execution. No servers to manage or pay for while idle; scales from zero to millions automatically. AWS Lambda, Cloudflare Workers, Vercel Functions.	<u>4.7 Shipping code safely</u>
<u>session ID</u>	The server's record that a particular user has logged in, identified by a long random session ID that is stored in the user's cookie. On each request the server looks up the ID to learn who is asking. Sessions expire after a period or on logout.	<u>5.3 Staying logged in: cookies, sessions,</u>
<u>single point of failure</u>	Any single component whose failure takes the whole system down. Removing single points of failure by adding duplicates is the main way to increase uptime.	<u>4.3 Step 2: give the database its own mac</u>
<u>single sign-on</u>	Single Sign-On: logging in once with a central identity provider (Google Workspace, Microsoft Entra, Okta)	<u>5.6 Security basics every product owner</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
	and being recognised by many applications without separate passwords. Built on OpenID Connect or the older SAML standard; a standard requirement for selling software to companies.	
<u>source code</u>	The human-readable text of a program, as written in a programming language and stored in files such as app.py or index.js. It is what developers edit, review, and keep in version control.	<u>1.1 What a programming language is, and</u>
<u>SQL</u>	Structured Query Language, the standard language for asking questions of and making changes to a relational database. Statements read almost like English: SELECT, FROM, WHERE, INSERT, UPDATE, DELETE, JOIN.	<u>3.4 SQL by example</u>
<u>SQL injection</u>	An attack in which text a user types (in a form or URL) is pasted into a SQL query and ends up being run as part of the query, letting the attacker read or destroy data. Prevented by	<u>5.7 Beyond the browser: mobile, desktop</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
	parameterised queries, which every modern framework and ORM uses by default.	
<u>stateless</u>	A server that keeps no memory of previous requests between one request and the next. Any copy can handle any request because everything that must be remembered (who is logged in, what is in the cart) is stored in a shared database or cache, not on the server itself.	<u>4.5 Step 4: caches, CDNs, and storage</u>
<u>static website</u>	A website made of fixed files (HTML, CSS, JavaScript, images) that are delivered to every visitor exactly as stored. Nothing is computed per visitor and no data is saved on the server.	<u>2.5 How the internet delivers a page</u>
<u>status code</u>	The three-digit number in an HTTP response that says how the request went. 200 means success, 301 means moved, 404 means not found, 500 means the server hit an error.	<u>2.5 How the internet delivers a page</u>
<u>syntax</u>	The grammar rules of a programming	<u>1.1 What a programming language is, and</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
	language: which symbols, words, and arrangements are valid. A syntax error is a sentence the language cannot parse, like a missing bracket.	
T		
<u>tables</u>	In a relational database, a grid holding all records of one kind of thing. Each row is one record (one customer) and each column is one attribute (name, email).	<u>3.3 Why every product needs a database</u>
<u>tags</u>	A label in angle brackets, such as <h1> or <p>, that HTML uses to mark what a piece of content is. Most tags come in pairs: an opening tag and a closing tag with a slash.	<u>2.3 A web page is three languages</u>
<u>templates</u>	An HTML file with placeholders (such as {{ customer_name }}) that the back end fills in with real data before sending the page. Templates keep page design separate from program logic.	<u>3.2 APIs and JSON: how programs talk to</u>
<u>third-party API</u>	An API offered by another company that your product calls to get a capability without	<u>3.3 Why every product needs a database</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
	building it: payments (Stripe), maps (Google), text messages (Twilio), AI (Claude API). Usually paid per use.	
<u>three-tier architecture</u>	The classic structure of a web application in three layers: the presentation tier (front end in the browser or app), the application or logic tier (back-end program), and the data tier (database). Each tier can be built, scaled, and replaced separately.	<u>3.6 Putting it together: the three-tier arc</u>
<u>token</u>	A string handed to a client after it authenticates, which the client then attaches to each request as proof of identity, instead of sending a password every time. Session IDs, API keys, JSON Web Tokens (JWTs), and OAuth access tokens are all kinds of tokens.	<u>5.4 HTTP Basic Authentication and API ke</u>
<u>transaction</u>	A group of database changes that either all succeed or all fail together, so the data is never left half-updated. Moving money between two accounts is the classic example.	<u>3.3 Why every product needs a database</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
<u>TypeScript</u>	A superset of JavaScript that adds type checking, so many mistakes are caught before the code runs. Compiles to plain JavaScript. The default choice for large front-end and Node.js projects.	<u>1.2 Fit for purpose: what each language v</u>
U		
<u>Uptime</u>	The fraction of time a service is working and reachable, usually quoted as a percentage. 99.9% uptime ('three nines') allows about 9 hours of downtime a year; 99.99% allows about 53 minutes.	<u>4.2 Step 1: everything on one server</u>
<u>URL</u>	Uniform Resource Locator, the full address of something on the web. It names the protocol (https), the domain (which server), and the path (which page or file on that server).	<u>2.5 How the internet delivers a page</u>
V		
<u>Version control</u>	A system that records every change to a project's code, who made it, and why, so that changes can be reviewed, combined,	<u>4.7 Shipping code safely</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
	and undone. Git is the universal tool; GitHub and GitLab host Git repositories online for teams.	
<u>Vertical scaling</u>	Adding capacity by making one machine bigger: more processor cores, more memory, faster disks. Simple, but has a hard ceiling and the machine remains a single point of failure.	<u>4.4 Step 3: many web servers behind a load balancer</u>
<u>virtual machine</u>	A program that behaves like a standardised imaginary computer and runs bytecode on top of whatever real platform it is installed on. The Java Virtual Machine (JVM) and the .NET runtime let one compiled program run unchanged on Windows, macOS, and Linux.	<u>1.6 Platforms and portability</u>
<u>virtual private servers</u>	Virtual Private Server: a rented slice of a physical server in a data center that behaves like your own dedicated computer, with its own operating system, IP address, and disk. The cheapest way to run a back end on	<u>4.2 Step 1: everything on one server</u>

KEYWORD	SHORT DEFINITION	WHERE IT IS INTRODUCED
	the internet; typically 5 to 50 dollars a month.	
W		
<u>web browser</u>	A program such as Chrome, Safari, Firefox, or Edge that requests web pages from servers and turns the HTML, CSS, and JavaScript it receives into what you see on screen.	<u>2.3 A web page is three languages</u>
<u>web server</u>	A program that listens for web requests and sends back web pages and files. Popular examples are Nginx and Apache. The word also refers to the computer running that program.	<u>2.2 Clients, servers, and programs</u>

Review Questions and Answers

The review questions from the end of each session, repeated here with model answers. Answers are written at the level of detail expected from a student; a shorter answer that captures the same ideas is fine.

Session 1: Choosing a programming language

1. Explain the difference between a compiler and an interpreter, and name one language of each kind.

A compiler translates the whole program into machine code ahead of time, producing an executable tied to one processor and operating system (C, C++, Rust, Go, Swift). An interpreter reads the source code and carries out its instructions on the spot, so the same file runs wherever the interpreter is installed (Python, JavaScript, Ruby). Compiled code runs faster; interpreted code is more portable and quicker to develop.

2. Why would nobody build a website in SQL, even though SQL is an excellent language?

SQL is a domain-specific language designed only for asking questions of and making changes to relational databases. It has no way to express page layout, respond to a click, send an email, or run general logic. Websites need general-purpose languages (JavaScript in the browser, Python or similar on the server); SQL is used alongside them for the database part.

3. Give three reasons a language's popularity matters to a business, then explain the new reason it matters in the age of AI.

Any three of: easier and cheaper hiring; every problem has been solved and written up publicly; long-term support and longevity of the product; faster security fixes because many people inspect the code; better tools and hosting support. The new reason: AI assistants learn from public code, so they write mainstream languages far more reliably than niche or very new ones. Building with AI makes mainstream choices more valuable, not less.

4. What is a package manager, and what is a hallucinated package?

A package manager (pip or uv for Python, npm for JavaScript) downloads and installs libraries from a central registry and records which ones a project depends on. A hallucinated package is a library an AI assistant confidently names that does not actually exist (or no longer exists in

that form); the install or import fails, which is why a person should verify that suggested dependencies are real and maintained.

5. A colleague insists your new online store must be written in C++ "because it is the fastest language." How do you respond?

Most web applications spend their time waiting for the database, network, and disk, not computing, so the language's raw speed is invisible next to those delays; companies such as Instagram and Spotify serve huge audiences from Python. Developer time is far more expensive than server time, and the architecture (caching, more servers) buys more speed than a faster language. The sensible approach is to build in a productive language, measure, and rewrite only a proven hot spot in a faster language, which AI assistance now makes cheap. C++ is the right choice for embedded devices, games, and trading systems, not for an online store.

6. Describe the three ways software reaches Windows, Mac, and Linux, and name a language for each.

(1) Compile separately for each platform, producing a different executable per processor and operating system (C, C++, Rust, Go, Swift). (2) Compile once to bytecode and run it on a virtual machine installed on each platform, Java's "write once, run anywhere" (Java, Kotlin, C# on .NET). (3) Ship the source code unchanged and run it with an interpreter installed on each platform (Python, JavaScript, Ruby); JavaScript's interpreter is built into every browser, making the web the most portable platform.

7. Why do companies often write their mobile app twice, and what alternative exists?

Apple and Google each built their platform around a preferred, compiled language with its own libraries and tools: Swift for iOS and Kotlin for Android. The best experience on each platform comes from using its native language, so a company wanting the best app on both writes two. The alternative is a cross-platform framework such as React Native or Flutter, which produces both apps from one codebase at some cost in performance and platform polish.

8. Which factor in language choice becomes *less* important with AI-assisted development, which becomes *more* important, and why?

Less important: the team's own familiarity with a language's syntax, and the labour of writing multiple platform versions, because the assistant can write in a language you do not know and port code between platforms. More important: popularity and maturity, because the assistant's reliability depends on how much public code exists in that language and version. Fit for purpose, ecosystem, and the physics of performance are unchanged, and testing on every platform still requires people.

Session 2: From a file to a page on the internet

1. In the client-server model, which side starts every conversation, and why does that matter for how servers are designed?

The client always starts. The server only waits and answers. This means a server must be always on and reachable, and it never needs to "find" its users; it just has to handle whatever arrives. It also means a server cannot push you information you did not ask for without tricks such as the browser repeatedly asking.

2. A web page is written in up to three languages. Name each one and state in one phrase what it is responsible for.

HTML describes the content and structure (headings, paragraphs, images, links). CSS controls appearance (colors, fonts, spacing, layout). JavaScript controls behaviour and interactivity (reacting to clicks, changing the page, fetching data).

3. Why can you open a static website from a folder on your desktop with no internet connection, but your friend across town cannot see it?

A static site is just files, and a browser can read files straight off the local disk (the address begins with `file:///`). Your friend's browser cannot reach files on your computer because your machine is not a publicly addressable, always-on server. To share it, the files must be placed on a hosted server with a domain name pointing to it.

4. Break the URL `https://shop.example.com/products/42?color=blue` into its parts and say what each part tells the browser.

`https` is the protocol (talk securely using HTTP). `shop.example.com` is the domain name, identifying which server to contact (`shop` is a subdomain of `example.com`). `/products/42` is the path, identifying which resource on that server (product number 42). `?color=blue` is the query string, an extra detail passed as a name=value pair.

5. What does DNS do, and what everyday object is it usually compared to?

DNS translates a human-readable domain name into the numeric IP address of the server hosting it. It is compared to a phone book or contacts list: you look up a name and get a number you can actually dial.

6. A customer reports seeing "500 Internal Server Error" while another reports "404 Not Found." Which one suggests a bug in your own code, and which suggests a bad link or typo?

500 means the server's own program failed while handling the request, so it points to a bug or outage on your side. 404 means the server is fine but no such page exists at that path, which usually indicates a broken link, a deleted page, or a typo in the address.

7. Give two things a purely static website cannot do, and explain in one sentence what has to be added to do them.

Any two of: remember information a visitor submits, personalise pages per visitor, keep a secret such as a payment key, or perform work such as sending emails or charging cards. To do them you add a back end: a program on the server that runs per request, applies logic, and reads and writes stored data.

8. Your Python course teaches you to write programs. In a web application, where would that Python code run, and why not in the visitor's browser?

Python runs on the server as back-end code (or on your own computer as a script). It does not run in the browser because browsers only execute JavaScript; that is the one language every browser was built to understand.

[Back to the Session 2 review box](#)

Session 3: Dynamic websites, APIs, and databases

1. In the Flask example, what does a route do, and what is the difference between the `/` route and the `/hours` route from the visitor's point of view?

A route connects a URL path to a function in the back end; when a request for that path arrives, the function runs and its return value becomes the response. The `/` route returns the same HTML every time, so it behaves like a static page. The `/hours` route checks the current time and returns different HTML depending on the answer, which is what makes it dynamic.

2. Explain what an API is using the restaurant analogy, then give a real example of a third-party API a small business might call and why.

An API is like a restaurant menu: a fixed list of what you may ask for and what you will receive, which lets outsiders use the kitchen without seeing inside it. Example: calling Stripe's API to charge a credit card, because building payment processing yourself is expensive, risky, and heavily regulated. Other valid answers: Twilio for text messages, Google Maps for addresses, the Claude API for text generation.

3. What is JSON, and why did it become the standard format for APIs?

JSON is a text format for data: curly braces hold named values, square brackets hold lists, text is quoted, numbers and true/false stand alone. It became the standard because it is human-readable, compact, maps directly onto the data structures of nearly every programming language (a Python dictionary, for instance), and is natively understood by JavaScript in the browser.

4. Match each HTTP method to a CRUD operation and give a bakery example for each.

POST = Create (place an order: `POST /api/orders`). GET = Read (list products: `GET /api/products`). PUT or PATCH = Update (mark an order ready: `PATCH /api/orders/88`). DELETE = Delete (cancel an order: `DELETE /api/orders/88`).

5. Give three reasons a business stores its data in a database rather than in ordinary files.

Any three of: many users and programs can write at the same time without corrupting the data; searches over millions of records are fast thanks to indexes; transactions guarantee changes are all-or-nothing so data is never left half-saved; complex questions (reports) can be asked in one line of SQL; constraints such as "every order must have a customer" are enforced automatically.

6. In the bakery schema, why does the orders table store a `customer_id` instead of the customer's name and email? Which term describes that column?

Storing the ID avoids copying the same customer details into every order. If Dana changes her email, it is updated once in the customers table and every order automatically reflects it; copies would drift out of sync. A column that holds another table's primary key is a foreign key.

7. In plain English, what does this query return?

```
SELECT customers.name, COUNT(*) FROM orders JOIN customers ON orders.customer_id =  
customers.customer_id GROUP BY customers.name;
```

For each customer who has placed at least one order, it returns the customer's name and the number of orders they have placed. The JOIN attaches the customer's name to each order via the `customer_id` key, and GROUP BY with COUNT tallies the orders per name.

8. What is dangerous about the statement `DELETE FROM orders;` and what single addition would make it safe?

It has no `WHERE` clause, so it deletes every row in the orders table, not just one. Adding a `WHERE` condition such as `WHERE order_id = 89` restricts it to the intended row. This is also why production databases are backed up and why write permissions are restricted.

9. Name the three tiers of the three-tier architecture and state which tier enforces the rule "customers get 10% off orders over \$50" and why it must be there.

Presentation tier (front end in the browser or app), application tier (back-end program), and data tier (database). The discount rule belongs in the application tier, on the server. Anything in the browser can be altered by the user, so a rule enforced only in the front end could be bypassed to get a discount on any order.

Session 4: Hosting, scaling, and the cloud

1. What does "deploy" mean, and why do teams put a staging environment between development and production?

To deploy is to put a version of the software onto servers where real users can reach it. Staging is a private replica of production where the team checks a release with realistic data and settings before customers see it, catching the differences between "works on my machine" and the live system.

2. Describe the Step 1 single-server deployment and name its most serious weakness.

One rented machine (a VPS running Linux) hosts the web server program, the back-end application, and the database together. Its most serious weakness is that it is a single point of failure: a disk failure, power loss, or a botched update takes down the website and the data at the same time. It also suffers from resource contention and has a hard capacity ceiling.

3. Give two reasons for moving the database onto its own server, and explain what "private network" means in that design.

Reasons (any two): each machine can be sized for its own job (databases want memory and fast disks); security improves because the database is no longer on the machine exposed to the internet; the web server can be rebuilt without touching data; it enables running several web servers that share one database. A private network is one with no route from the public internet: only the company's own servers can connect to the database, enforced by firewall rules.

4. Explain the difference between vertical and horizontal scaling. Which one does modern web architecture rely on, and why?

Vertical scaling makes one machine bigger (more processor, memory). Horizontal scaling adds more identical machines and spreads work across them. Modern architecture relies on horizontal scaling because it has no hard ceiling, tolerates the failure of any single machine, and lets capacity grow and shrink with demand, whereas a single big machine remains a single point of failure and eventually cannot be made bigger.

5. What does a load balancer do, and what is a health check?

A load balancer holds the public address, receives every incoming request, and distributes requests across the pool of web servers so none is overloaded. A health check is a small request it sends to each server every few seconds; a server that fails to respond correctly is removed from rotation until it recovers, so customers never reach a broken server.

6. Why must web servers be stateless in a load-balanced setup? Where does the "state" go instead?

Because the load balancer may send a user's consecutive requests to different servers, any server that remembered something locally (such as "this person is logged in") would leave the others ignorant of it, and the site would appear broken. Servers therefore keep nothing between requests; who is logged in, cart contents, and all other shared facts live in a shared store every server can reach, such as the database or a Redis session store.

7. A customer in Tokyo complains that images load slowly from your site hosted in Virginia. Which component fixes this and how?

A content delivery network (CDN). It keeps copies of static files (images, CSS, JavaScript, video) on servers in cities around the world, so the Tokyo customer receives the image from a nearby server in a few milliseconds instead of waiting for it to cross the Pacific. It also removes that traffic from your own servers.

8. What is a cache, and what is the one hard problem every cache has?

A cache is a fast temporary copy of data that was expensive to compute or fetch, kept so the next request for it is answered instantly (for example a product list stored in memory instead of re-running the SQL query). The hard problem is staleness: the copy can be out of date, so someone must decide how long it may live and when it must be refreshed or discarded.

9. Place each of these on the IaaS / PaaS / SaaS / serverless menu: renting a virtual machine on AWS EC2; using Shopify to run a store; connecting a GitHub repo to Render so it runs your code; running a small function on AWS Lambda when a file is uploaded.

EC2 virtual machine = IaaS (you manage everything on it). Shopify = SaaS (finished software by subscription). Render running your code = PaaS (you provide code, they run the servers). Lambda function on upload = serverless (a single function run and billed per event).

10. You are deploying a prototype built with Claude Code for a business school project. Which layer of the cloud menu is the sensible choice and why?

PaaS (Render, Railway, Vercel, Fly.io) with a managed database (Supabase, Neon). It provides HTTPS, deployment from GitHub, load balancing, and horizontal scaling with no servers to administer, for little or no money, which matches a small team with no operations expertise. IaaS adds work without benefit at this scale.

[Back to the Session 4 review box](#)

Session 5: Identity, security, and software beyond the browser

1. Define authentication and authorization, and give an example where a system gets the first right and the second wrong.

Authentication verifies who a user is (password, code, fingerprint, Google login). Authorization decides what that identified user may do. Example of getting only the first right: a site that checks you are logged in but lets you view any order by changing the number in the URL, so any customer can read any other customer's order.

- 2. Why should a database store a hash of each password rather than the password itself? What does it mean if a company can email you your old password?**

A hash is a one-way scramble that cannot be reversed to recover the password, so if the database is stolen the attackers cannot read users' passwords (which people reuse elsewhere). Login works by hashing what the user types and comparing. If a company can send you your old password, it is storing passwords in readable form, which is a serious security failure.

- 3. HTTP is stateless, yet you stay logged in. Explain the role of the cookie and the session store.**

On login the server creates a session record ("ID a8f3... = Dana, expires in 14 days") in a shared session store and tells the browser to save the ID in a cookie. The browser automatically sends the cookie with every later request; the server looks up the ID and knows who is asking. Any web server in the pool can do this lookup, which is what allows servers to be stateless.

- 4. What does HTTP Basic Authentication send, in what form, and why is it acceptable only over HTTPS?**

It sends the username and password on every request in an Authorization header, encoded in Base64 (for example Base64 of "dana:correct-horse"). Base64 is encoding, not encryption, and is instantly reversible, so over plain HTTP the password would be readable by anyone on the network. HTTPS encrypts the whole exchange, which is the only thing keeping the credentials private.

- 5. Name the four roles in OAuth and identify each one when a scheduling app asks to read your Google Calendar.**

Resource owner: you, the user who owns the calendar. Client: the scheduling app requesting access. Authorization server: Google's login and consent system, which authenticates you and issues the token. Resource server: the Google Calendar API that holds the data the token unlocks.

- 6. In the OAuth flow, why does Google send a one-time code back through the browser instead of the access token itself? And why does the user never type their Google password into the bakery's app?**

The code is useless on its own: it expires quickly and can only be exchanged for a token by presenting the client secret, which lives on the bakery's server. Exchanging it in a server-to-server call means the token never passes through the browser where it could be intercepted or

leaked. The user types their password only at Google so the bakery never possesses it; the bakery receives a scoped, expiring, revocable token instead, which limits damage if the app is careless or compromised.

7. What is a scope in OAuth, and where does the user see it?

A scope is a named, limited permission the application requests, such as "read your email address" or "read calendar events." The user sees the requested scopes on the consent screen at the authorization server (Google) before approving, and the issued token can do nothing beyond them.

8. What is SQL injection, and why does using a modern framework or ORM largely prevent it?

SQL injection occurs when text a user types is pasted directly into a SQL statement so that it runs as part of the query; typing `' OR 1=1 --` into a login form could make the query match every user. Modern frameworks and ORMs use parameterised queries, which send user input to the database separately from the SQL text so it is always treated as data, never as code.

9. Your company has a web app with a working API. The CEO wants "an iPhone app and an Android app." In architectural terms, how big is that project, and what alternative might you propose first?

Because the back end and API already exist, the apps are new front ends rather than a new system: the mobile apps call the same API and use the same data and login. It is still real work (two native codebases or one cross-platform codebase, app store review, updates). A cheaper first step is a progressive web app, which is installable to the home screen from the existing web app with no store involvement, unless the product needs native-only features such as rich offline use or certain hardware access.

10. Where in a product's architecture does a call to an AI model such as Claude belong, and why must it not be made from the browser?

In the back end (application tier), as a call to a third-party API, after the back end has checked who is asking and gathered relevant data from the database to include in the prompt. It must not be made from the browser because the API key would then be visible to every visitor, who could use it to run up the company's bill, and because the business rules and private data added to the prompt must stay on the server.

[Back to the Session 5 review box](#)

Suggested Follow-Up Activities

- **Architecture sketch.** In pairs, students pick a familiar product (a food-delivery app, a ticketing site, a fitness tracker with a phone app and a wearable) and draw its architecture using only the boxes from this module: front ends, load balancer, web servers, database, cache, CDN, object storage, third-party APIs, AI model. Present and critique.
- **Read a real API.** Open the public documentation for Stripe, Shopify, or the Claude API. Identify three endpoints, their HTTP methods, and the JSON they return. Find where authentication is described and name the mechanism (API key, OAuth, or both).
- **Consent-screen audit.** Students review the third-party apps connected to their own Google or Microsoft account, list the scopes each was granted, and revoke any they no longer use. Discuss what each scope allowed.
- **Deploy something.** With Claude Code, build the bakery's Flask app with an SQLite database, then deploy it to a free PaaS tier. Students document which cloud-menu layer they used, what the platform handled for them, and what the monthly cost would be at 10× traffic.
- **Vendor conversation role-play.** One student plays a founder, another an agency proposing a build. The founder must ask at least five questions from this module (where does the database live, can we run two copies, who holds the API keys, do we support Sign in with Google, what is the uptime target) and judge the answers.